

Anthropic Developer Certification Practice Exam

Anthropic Developer Certification Practice Exam (Attempt 2)

Total questions: 60

Question 1

Topic: Custom Slash Commands | Difficulty: easy | Scenario: 11 | ID: 599

You want to create a custom `/review` slash command that runs your team's standard code review checklist. This command should be available to every developer when they clone or pull the repository. Where should you create this command file?

- **A)** In the `.claude/commands/` directory in the project repository
- **B)** In `~/.claude/commands/` in each developer's home directory
- **C)** In the `CLAUDE.md` file at the project root
- **D)** In a `.claude/config.json` file with a `commands` array

Correct Answer: A - In the `.claude/commands/` directory in the project repository

Explanation: Project-scoped custom slash commands should be stored in the `.claude/commands/` directory within the repository. These commands are version-controlled and automatically available to all developers when they clone or pull the repo. Option B (`~/.claude/commands/`) is for personal commands that aren't shared via version control. Option C (`CLAUDE.md`) is for project instructions and context, not command definitions. Option D describes a configuration mechanism that doesn't exist in Claude Code.

Option-level rationale: - A) Placing custom slash commands in the `.claude/commands/` directory within the project repository is correct because these files are version-controlled and automatically available to every developer who clones or pulls the repo. This is the designated location for project-scoped custom commands in Claude Code. - B) Storing commands in `~/.claude/commands/` in each developer's home directory creates personal, user-scoped commands that are not shared via version control. This approach would require each developer to manually set up the command, defeating the goal of automatic availability upon cloning or pulling the repository. - C) The `CLAUDE.md` file at the project root is used for project instructions, context, and conventions that guide Claude's behavior, not for defining custom slash commands. Command definitions require their own dedicated files in the appropriate commands directory. - D) A `.claude/config.json` file with a `commands` array is not a valid mechanism for defining custom slash commands in Claude Code. This configuration format does not exist, and commands must be created as individual files in the appropriate commands directory.

Question 2

Topic: CI/CD Integration | Difficulty: easy | Scenario: 16 | ID: 606

Your pipeline script runs `claude "Analyze this pull request for security issues"` but the job hangs indefinitely. Logs indicate Claude Code is waiting for interactive input. What's the correct approach to run Claude Code in an automated pipeline?

- **A)** Add the `-p` flag: `claude -p "Analyze this pull request for security issues"`
- **B)** Set the environment variable `CLAUDE_HEADLESS=true` before running the command
- **C)** Redirect stdin from `/dev/null`: `claude "Analyze this pull request for security issues" < /dev/null`
- **D)** Add the `--batch` flag: `claude --batch "Analyze this pull request for security issues"`

Correct Answer: A - Add the `-p` flag: `claude -p "Analyze this pull request for security issues"`

Explanation: The `-p` (or `--print`) flag is the documented way to run Claude Code in non-interactive mode. It processes the prompt, outputs the result to stdout, and exits without waiting for user input—exactly what CI/CD pipelines require. The other options reference non-existent features (`CLAUDE_HEADLESS` environment variable, `-batch` flag) or use Unix workarounds that don't properly address Claude Code's command syntax.

Option-level rationale: - A) The `-p` (or `--print`) flag is the documented way to run Claude Code in non-interactive mode. It processes the given prompt, outputs the result to stdout, and exits without waiting for user input, making it ideal for CI/CD pipelines. - B) The `CLAUDE_HEADLESS=true` environment variable is not a documented or supported feature of Claude Code. This approach would not switch Claude Code into a non-interactive mode and the job would still hang. - C) Redirecting stdin from `/dev/null` is a generic Unix workaround that does not properly address Claude Code's command syntax for non-interactive use. While it might prevent some input waiting, it is not the correct or reliable way to run Claude Code in a pipeline. - D) The `--batch` flag is not a documented or supported option for Claude Code. Using this non-existent flag would likely result in an error or be ignored, failing to resolve the interactive input issue.

Question 3

Topic: Plan Mode vs Direct Execution | Difficulty: easy | Scenario: 11 | ID: 607

You've been assigned to restructure the team's monolithic application into microservices. This will involve changes across dozens of files and requires decisions about service boundaries and module dependencies. Which approach should you take?

- **A)** Enter plan mode to explore the codebase, understand dependencies, and design an implementation approach before making changes.
- **B)** Start with direct execution and make changes incrementally, letting the implementation reveal the natural service boundaries.
- **C)** Use direct execution with comprehensive upfront instructions detailing exactly how each service should be structured.

- **D)** Begin in direct execution mode and only switch to plan mode if you encounter unexpected complexity during implementation.

Correct Answer: A - Enter plan mode to explore the codebase, understand dependencies, and design an implementation approach before making changes.

Explanation: Plan mode is designed for complex tasks involving large-scale changes, multiple valid approaches, and architectural decisions—exactly what monolith-to-microservices restructuring requires. It enables safe codebase exploration and design before committing to changes. Option B risks costly rework when dependencies are discovered late. Option C assumes you already know the right structure without exploring the code. Option D ignores that the complexity is already stated in the requirements, not something that might emerge later.

Option-level rationale: - A) Using plan mode to explore the codebase, understand dependencies, and design an approach before making changes is the correct strategy for a complex architectural restructuring like breaking apart a monolith. This allows safe exploration and informed decision-making about service boundaries before committing to potentially costly changes across dozens of files. - B) Starting with direct execution and making incremental changes without upfront planning risks costly rework when hidden dependencies and architectural conflicts are discovered late in the process. A monolith-to-microservices migration requires understanding the full dependency graph before making structural decisions, not discovering it piecemeal. - C) Providing comprehensive upfront instructions for exact service structure assumes you already know the right architecture without first exploring the existing codebase and its dependencies. This approach bypasses the critical discovery phase needed to make informed decisions about service boundaries in a monolithic application. - D) Starting in direct execution mode and only switching to plan mode if unexpected complexity arises ignores the fact that the task is already known to be complex—involving dozens of files, service boundary decisions, and module dependencies. The complexity is evident from the requirements themselves, not something that might emerge later.

Question 4

Production data shows that in 12% of cases, your agent skips `get_customer` entirely and calls `lookup_order` using only the customer's stated name, occasionally leading to misidentified accounts and incorrect refunds. What change would most effectively address this reliability issue?

- **A)** Add a programmatic prerequisite that blocks `lookup_order` and `process_refund` calls until `get_customer` has returned a verified customer ID.
- **B)** Enhance the system prompt to state that customer verification via `get_customer` is mandatory before any order operations.
- **C)** Add few-shot examples showing the agent always calling `get_customer` first, even when customers volunteer order details.
- **D)** Implement a routing classifier that analyzes each request and enables only the subset of tools appropriate for that request type.

Correct Answer: A - Add a programmatic prerequisite that blocks `lookup_order` and `process_refund` calls until `get_customer` has returned a verified customer ID.

Explanation: When a specific tool sequence is required for critical business logic (like verifying customer identity before processing refunds), programmatic enforcement provides deterministic guarantees that prompt-based approaches cannot. Options B and C rely on probabilistic LLM compliance, which is insufficient when errors have financial consequences. Option D addresses tool availability rather than tool ordering, which is not the actual problem.

Option-level rationale: - A) Adding a programmatic prerequisite that blocks downstream tools until `get_customer` returns a verified customer ID provides a deterministic guarantee that the required sequence is followed. This is the most effective approach because it removes the possibility of the agent skipping verification, regardless of LLM behavior. - B) Enhancing the system prompt to mandate customer verification relies on the LLM consistently following instructions, which is inherently probabilistic. Since 12% of cases already show the agent skipping this step, prompt-based guidance alone is insufficient for preventing errors with financial consequences. - C) Adding few-shot examples demonstrating the correct tool-calling sequence can improve compliance but still depends on the LLM generalizing from those examples. This approach cannot guarantee deterministic adherence, making it unreliable for a critical business process where mistakes lead to incorrect refunds. - D) Implementing a routing classifier to enable only relevant tool subsets addresses which tools are available per request type, not the ordering in which tools must be called. The core issue is

enforcing a required sequence (verification before order lookup), which tool-subset selection does not solve.

Question 5

Topic: Tool Interface Design | Difficulty: medium | Scenario: 1 | ID: 626

Production logs show the agent frequently calls `get_customer` when users ask about orders (e.g., "check my order #12345"), instead of calling `lookup_order`. Both tools have minimal descriptions ("Retrieves customer information" / "Retrieves order details") and accept similar identifier formats. What's the most effective first step to improve tool selection reliability?

- **A)** Add few-shot examples to the system prompt demonstrating correct tool selection patterns, with 5-8 examples showing order-related queries routing to `lookup_order`.
- **B)** Expand each tool's description to include input formats it handles, example queries, edge cases, and boundaries explaining when to use it versus similar tools.
- **C)** Implement a routing layer that parses user input before each turn and pre-selects the appropriate tool based on detected keywords and identifier patterns.
- **D)** Consolidate both tools into a single `lookup_entity` tool that accepts any identifier and internally determines which backend to query.

Correct Answer: B - Expand each tool's description to include input formats it handles, example queries, edge cases, and boundaries explaining when to use it versus similar tools.

Explanation: Tool descriptions are the primary mechanism LLMs use for tool selection. When descriptions are minimal, models lack the context to differentiate between similar tools. Option B directly addresses this root cause with a low-effort, high-leverage fix. Few-shot examples (A) add token overhead without fixing the underlying issue. A routing layer (C) is over-engineered and bypasses the LLM's natural language understanding. Consolidating tools (D) is a valid architectural choice but requires more effort than a "first step" warrants when the immediate problem is inadequate descriptions.

Option-level rationale: - A) Adding few-shot examples to the system prompt increases token overhead and addresses symptoms rather than the root cause. While examples can help, they are less effective and scalable than fixing the minimal tool descriptions that the LLM relies on for tool selection. - B) Expanding tool descriptions to include input formats, example queries, edge cases, and boundaries directly addresses the root cause—minimal descriptions that leave the LLM unable to distinguish between similar tools. This is a low-effort, high-leverage first step that improves the primary mechanism LLMs use for tool selection. - C) Implementing a keyword-based routing layer is over-engineered for a first step and bypasses the LLM’s natural language understanding capabilities. It also introduces brittle logic that must be maintained separately and may fail on ambiguous or novel queries. - D) Consolidating both tools into a single endpoint is a valid architectural approach but requires significantly more engineering effort than simply improving tool descriptions. As a first step, it is disproportionate when the immediate problem—inadequate descriptions—can be resolved much more quickly.

Question 6

Topic: Escalation Decisions | Difficulty: medium | Scenario: 1 | ID: 627

Your agent achieves 55% first-contact resolution, well below the 80% target. Logs show it escalates straightforward cases (standard damage replacements with photo evidence) while attempting to autonomously handle complex situations requiring policy exceptions. What’s the most effective way to improve escalation calibration?

- **A)** Add explicit escalation criteria to your system prompt with few-shot examples demonstrating when to escalate versus resolve autonomously.
- **B)** Have the agent self-report a confidence score (1-10) before each response and automatically route requests to humans when confidence falls below a threshold.
- **C)** Deploy a separate classifier model trained on historical tickets to predict which requests need escalation before the main agent begins processing.
- **D)** Implement sentiment analysis to detect customer frustration levels and automatically escalate when negative sentiment exceeds a threshold.

Correct Answer: A - Add explicit escalation criteria to your system prompt with few-shot examples demonstrating when to escalate versus resolve autonomously.

Explanation: Adding explicit escalation criteria with few-shot examples directly addresses the root cause: unclear decision boundaries. This is the proportionate first response before adding infrastructure. Option B fails because LLM self-reported confidence is poorly calibrated—the agent is already incorrectly confident on hard cases. Option C is over-engineered, requiring labeled data and ML infrastructure when prompt optimization hasn't been tried. Option D solves a different problem entirely; sentiment doesn't correlate with case complexity, which is the actual issue.

Option-level rationale: - A) Adding explicit escalation criteria with few-shot examples directly addresses the root cause—unclear decision boundaries between straightforward and complex cases. This is the most proportionate and effective first intervention, as it teaches the agent precisely when to escalate versus resolve autonomously without requiring additional infrastructure. - B) LLM self-reported confidence scores are notoriously poorly calibrated, and the logs already show the agent is incorrectly confident on complex cases while being overly cautious on simple ones. A numeric self-assessment would likely replicate the same miscalibration rather than fix the underlying decision boundary problem. - C) Deploying a separate classifier model is over-engineered for this situation, requiring labeled training data and additional ML infrastructure when the simpler approach of optimizing the system prompt with clear escalation criteria hasn't yet been attempted. This adds unnecessary complexity and latency when prompt-level improvements should be tried first. - D) Sentiment analysis detects customer frustration, not case complexity, so it addresses a fundamentally different problem than the one described. Straightforward cases with calm customers would still be escalated and complex edge cases with polite customers would still be handled autonomously, failing to fix the miscalibration issue.

Question 7

Topic: Batch Processing | Difficulty: medium | Scenario: 16 | ID: 631

Your team wants to reduce API costs for automated analysis. Currently, real-time Claude calls power two workflows: (1) a blocking pre-merge check that must complete before developers can merge, and (2) a technical debt report generated overnight for review the next morning. Your manager proposes switching both to the Message Batches API for its 50% cost savings. How should you evaluate this proposal?

- **A)** Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks.
- **B)** Switch both workflows to batch processing with status polling to check for completion.
- **C)** Keep real-time calls for both workflows to avoid batch result ordering issues.
- **D)** Switch both to batch processing with a timeout fallback to real-time if batches take too long.

Correct Answer: A - Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks.

Explanation: The Message Batches API offers 50% cost savings but has processing times up to 24 hours with no guaranteed latency SLA. This makes it unsuitable for blocking pre-merge checks where developers wait for results, but ideal for overnight batch jobs like technical debt reports. Option B is wrong because relying on “often faster” completion isn’t acceptable for blocking workflows. Option C reflects a misconception—batch results can be correlated using `custom_id` fields. Option D adds unnecessary complexity when the simpler solution is matching each API to its appropriate use case.

Option-level rationale: - A) This is the correct approach because the Message Batches API’s up to 24-hour processing time with no guaranteed latency SLA makes it ideal for overnight technical debt reports but unsuitable for blocking pre-merge checks where developers are waiting. This matches each workflow to the appropriate API based on its latency requirements. - B) Switching both workflows to batch processing is problematic because the Message Batches API has no guaranteed latency SLA and can take up to 24 hours, making it unsuitable for blocking pre-merge checks where developers must wait for results before merging. Status polling does not solve the fundamental issue of unpredictable completion times for a latency-sensitive workflow. - C) Keeping real-time calls for both workflows unnecessarily forgoes the 50% cost savings on the overnight technical debt reports, which have no latency constraints. The concern about batch result ordering is a misconception, as batch results can be correlated to their requests using `custom_id` fields. - D) Adding a timeout fallback introduces unnecessary architectural complexity when the simpler and more reliable solution is to use each API for its appropriate use case—real-time for latency-sensitive pre-merge checks and batch for overnight reports. This hybrid approach also still risks delays and inconsistent behavior for the pre-merge workflow.

Question 8

Topic: Task Decomposition | Difficulty: medium | Scenario: 16 | ID: 632

A pull request modifies 14 files across the stock tracking module. Your single-pass review analyzing all files together produces inconsistent results: detailed feedback for some files but superficial comments for others, obvious bugs missed, and contradictory feedback—flagging a pattern as problematic in one file while approving identical code elsewhere in the same PR. How should you restructure the review?

- **A)** Split into focused passes: analyze each file individually for local issues, then run a separate integration-focused pass examining cross-file data flow.
- **B)** Require developers to split large PRs into smaller submissions of 3-4 files before the automated review runs.
- **C)** Switch to a higher-tier model with a larger context window to give all 14 files adequate attention in one pass.
- **D)** Run three independent review passes on the full PR and only flag issues that appear in at least two of the three runs.

Correct Answer: A - Split into focused passes: analyze each file individually for local issues, then run a separate integration-focused pass examining cross-file data flow.

Explanation: Splitting reviews into focused passes directly addresses the root cause: attention dilution when processing many files at once. File-by-file analysis ensures consistent depth, while a separate integration pass catches cross-file issues. Option B shifts burden to developers without improving the system. Option C misunderstands that larger context windows don't solve attention quality issues—the “lost in the middle” problem persists. Option D would actually suppress detection of real bugs by requiring consensus on issues that may only be caught intermittently.

Option-level rationale: - A) Splitting the review into focused per-file passes directly addresses the root cause of attention dilution, ensuring consistent depth and catching local issues reliably. A separate integration-focused pass then handles cross-file concerns like data flow dependencies, covering both dimensions of review quality. - B) Requiring developers to break up PRs shifts the burden to the development workflow without actually improving the review system's analytical approach. This is a process workaround rather than a technical solution, and logically related changes across files may still need to be reviewed together for coherence. - C) Simply using a larger context

window does not solve the core problem of attention quality degradation; the “lost in the middle” phenomenon means models still struggle to give uniform attention across large inputs. The inconsistencies and contradictions observed are symptoms of attention dilution, not insufficient context capacity. - D) Running three full passes and only flagging issues found in at least two runs would actually suppress real bugs that are only intermittently detected, reducing recall. This consensus approach treats the symptom of inconsistency by filtering out results rather than addressing the underlying cause of uneven attention.

Question 9

Topic: Multi-Agent Orchestration | Difficulty: easy | Scenario: 3 | ID: 634

After running the system on the topic “impact of AI on creative industries,” you observe that each subagent completes successfully: the web search agent finds relevant articles, the document analysis agent summarizes papers correctly, and the synthesis agent produces coherent output. However, the final reports cover only visual arts, completely missing music, writing, and film production. When you examine the coordinator’s logs, you see it decomposed the topic into three subtasks: “AI in digital art creation,” “AI in graphic design,” and “AI in photography.” What is the most likely root cause?

- **A)** The synthesis agent lacks instructions for identifying coverage gaps in the findings it receives from other agents.
- **B)** The coordinator agent’s task decomposition is too narrow, resulting in subagent assignments that don’t cover all relevant domains of the topic.
- **C)** The web search agent’s queries are not comprehensive enough and need to be expanded to cover more creative industry sectors.
- **D)** The document analysis agent is filtering out sources related to non-visual creative industries due to overly restrictive relevance criteria.

Correct Answer: B - The coordinator agent’s task decomposition is too narrow, resulting in subagent assignments that don’t cover all relevant domains of the topic.

Explanation: The coordinator’s logs reveal the root cause directly: it decomposed “creative industries” into only visual arts subtasks (digital art, graphic design,

photography), completely omitting music, writing, and film. The subagents executed their assigned tasks correctly—the problem is what they were assigned. This makes Option B clearly correct: the coordinator’s task decomposition was too narrow. Options A, C, and D incorrectly blame downstream agents that are working correctly within their assigned scope. The synthesis agent can only combine findings it receives; the web search agent searched exactly what it was assigned; and there’s no evidence of document filtering—the missing industries were never researched in the first place.

Option-level rationale: - A) While the synthesis agent could theoretically flag coverage gaps, it can only work with the findings it receives. The root problem is upstream—the coordinator never assigned subtasks for music, writing, or film, so there were no findings about those domains to flag as missing. - B) The coordinator’s logs directly reveal it decomposed the broad topic into only three visual arts subtasks (digital art, graphic design, photography), completely omitting music, writing, and film. Since the subagents all executed their assigned tasks correctly, the narrow decomposition by the coordinator is clearly the root cause of the missing coverage. - C) The web search agent correctly found relevant articles for the subtasks it was assigned. The problem is not with the search queries but with the fact that the coordinator never assigned subtasks covering music, writing, or film production in the first place. - D) There is no evidence that the document analysis agent filtered out non-visual sources. The missing coverage is explained entirely by the coordinator’s narrow task decomposition, which never directed any subagent to research non-visual creative industries.

Question 10

Topic: Error Propagation | Difficulty: easy | Scenario: 3 | ID: 640

The web search subagent times out while researching a complex topic. You need to design how this failure information flows back to the coordinator agent. Which error propagation approach best enables intelligent recovery?

- **A)** Return structured error context to the coordinator including the failure type, the attempted query, any partial results, and potential alternative approaches.
- **B)** Implement automatic retry logic with exponential backoff within the subagent, returning a generic “search unavailable” status only after all retries are exhausted.

- **C)** Catch the timeout within the subagent and return an empty result set marked as successful.
- **D)** Propagate the timeout exception directly to a top-level handler that terminates the entire research workflow.

Correct Answer: A - Return structured error context to the coordinator including the failure type, the attempted query, any partial results, and potential alternative approaches.

Explanation: Structured error context gives the coordinator the information it needs to make intelligent recovery decisions—whether to retry with a modified query, try an alternative approach, or proceed with partial results. Option B's generic status hides valuable context from the coordinator, preventing informed decisions. Option C suppresses the error by marking failure as success, which prevents any recovery and risks incomplete research outputs. Option D terminates the entire workflow unnecessarily when recovery strategies could succeed.

Option-level rationale: - A) Returning structured error context—including the failure type, attempted query, partial results, and alternative approaches—gives the coordinator all the information it needs to make intelligent recovery decisions, such as retrying with a modified query or proceeding with partial results. This is the best approach because it preserves maximum context for informed decision-making at the coordination level. - B) While automatic retries with exponential backoff can be useful, returning only a generic "search unavailable" status after exhausting retries hides valuable context from the coordinator, preventing it from making informed recovery decisions such as modifying the query or trying alternative research approaches. The coordinator needs detailed failure information to orchestrate intelligent recovery, not just a binary success/failure status. - C) Catching the timeout and returning an empty result set marked as successful suppresses the error entirely, preventing the coordinator from knowing a failure occurred. This risks producing incomplete or misleading research outputs because the coordinator cannot trigger any recovery strategy for a problem it doesn't know exists. - D) Propagating the timeout exception directly to a top-level handler that terminates the entire workflow is an unnecessarily drastic response, as the coordinator could potentially recover through alternative strategies like modifying the query or using cached/partial results. This approach sacrifices resilience by treating a single subagent failure as a fatal workflow error.

Question 11

Topic: Tool Distribution | Difficulty: hard | Scenario: 3 | ID: 645

During testing, you observe that the synthesis agent frequently needs to verify specific claims while combining findings. Currently, when verification is needed, the synthesis agent returns control to the coordinator, which invokes the web search agent, then re-invokes synthesis with results. This adds 2-3 round trips per task and increases latency by 40%. Your evaluation shows that 85% of these verifications are simple fact-checks (dates, names, statistics) while 15% require deeper investigation. What's the most effective approach to reduce overhead while maintaining system reliability?

- **A)** Give the synthesis agent a scoped `verify_fact` tool for simple lookups, while complex verifications continue delegating to the web search agent through the coordinator.
- **B)** Have the synthesis agent accumulate all verification needs and return them as a batch to the coordinator at the end of its pass, which then sends them all to the web search agent at once.
- **C)** Give the synthesis agent access to all web search tools so it can handle any verification need directly without round-trips through the coordinator.
- **D)** Have the web search agent proactively cache extra context around each source during initial research, anticipating what the synthesis agent might need to verify.

Correct Answer: A - Give the synthesis agent a scoped `verify_fact` tool for simple lookups, while complex verifications continue delegating to the web search agent through the coordinator.

Explanation: Option A applies the principle of least privilege by giving the synthesis agent only what it needs for the 85% common case (simple fact verification) while preserving the existing coordination pattern for complex cases. Option B's batching approach creates blocking dependencies since synthesis steps may depend on earlier verified facts, making all-at-once batching impractical. Option C over-provisions the synthesis agent, violating separation of concerns and potentially degrading its accuracy on its primary synthesis task. Option D relies on speculative caching that cannot reliably predict what the synthesis agent will need to verify.

Option-level rationale: - A) Providing a scoped fact-verification tool handles the 85% of simple lookups directly, eliminating most round-trips while preserving the

coordinator-based delegation path for the 15% of complex verifications. This applies the principle of least privilege, keeping the synthesis agent focused on its primary task while still reducing latency significantly. - B) Batching all verification needs until the end of the synthesis pass creates blocking dependencies, since later synthesis steps may depend on facts that needed to be verified earlier. This approach cannot handle sequential dependencies within the synthesis process and would likely degrade output quality. - C) Giving the synthesis agent full access to all web search tools violates separation of concerns and over-provisions it beyond what is needed, potentially degrading its performance on its primary synthesis task. This approach also increases complexity and the risk of misuse of powerful search capabilities. - D) Proactive caching relies on speculative prediction of what the synthesis agent will need to verify, which is inherently unreliable and wastes resources fetching information that may never be needed. This approach cannot anticipate the specific facts requiring verification with sufficient accuracy to meaningfully reduce round-trips.

Question 12

Topic: Multi-Agent Orchestration | Difficulty: medium | Scenario: 3 | ID: 646

During testing, combined outputs from the web search agent (85K tokens including page content) and the document analysis agent (70K tokens including reasoning chains) total 155K tokens, but the synthesis agent performs optimally with inputs under 50K tokens. What's the most effective solution?

- **A)** Modify upstream agents to return structured data (key facts, citations, relevance scores) instead of verbose content and reasoning
- **B)** Add an intermediate summarization agent that condenses findings before passing to synthesis
- **C)** Have the synthesis agent process findings in sequential batches, maintaining running state between calls
- **D)** Store findings in a vector database and give the synthesis agent retrieval tools to query during its work

Correct Answer: A - Modify upstream agents to return structured data (key facts, citations, relevance scores) instead of verbose content and reasoning

Explanation: Option A addresses the root cause by having upstream agents output only what downstream agents need—structured facts, citations, and relevance scores rather than verbose content and reasoning traces. This reduces tokens at the source while preserving essential information. Option B adds latency, cost, and potential information loss through another agent that itself must process 155K tokens. Option C degrades synthesis quality since sequential batching prevents seeing all findings together to identify patterns and contradictions. Option D is over-engineered for this use case—RAG retrieval works for large knowledge bases, but synthesis requires comprehensive coverage of known inputs, not selective retrieval based on queries.

Option-level rationale: - A) Modifying upstream agents to return structured data (key facts, citations, relevance scores) addresses the root cause by reducing token volume at the source while preserving essential information. This eliminates verbose page content and reasoning chains that inflate token counts without adding value for the synthesis step. - B) Adding an intermediate summarization agent introduces additional latency, cost, and risk of information loss, while the summarization agent itself must still process the full 155K tokens. This treats the symptom rather than the root cause of upstream agents producing unnecessarily verbose output. - C) Processing findings in sequential batches degrades synthesis quality because the agent cannot see all findings simultaneously to identify cross-source patterns, contradictions, and connections. Maintaining running state between calls also adds complexity and risks compounding information loss across batches. - D) Using a vector database with retrieval tools is over-engineered for this scenario, as synthesis requires comprehensive coverage of all known findings rather than selective query-based retrieval. This approach risks missing important information that doesn't match the synthesis agent's queries and adds unnecessary architectural complexity.

Question 13

Topic: Tool Selection Reliability | Difficulty: hard | Scenario: 1 | ID: 654

Your agent handles single-concern requests with 94% accuracy (e.g., "I need a refund for order #1234"). However, when customers include multiple concerns in one message (e.g., "I need a refund for order #1234 and also want to update my shipping address for order #5678"), tool selection accuracy drops to 58%. The agent typically addresses

only one concern or mixes up parameters between requests. What's the most effective approach to improve reliability for multi-concern requests?

- **A)** Implement a preprocessing layer that uses a separate model call to decompose multi-concern messages into individual requests, process each independently, then combine the results.
- **B)** Add few-shot examples to your prompt demonstrating the correct reasoning and tool sequence for multi-concern requests.
- **C)** Consolidate related tools into fewer, more general-purpose tools.
- **D)** Implement response validation that detects incomplete responses and automatically re-prompts the agent to address any missed concerns.

Correct Answer: B - Add few-shot examples to your prompt demonstrating the correct reasoning and tool sequence for multi-concern requests.

Explanation: Few-shot examples are the most effective first approach because the agent already handles single-concern requests well (94% accuracy), indicating it understands the tools—it just needs guidance on the multi-concern pattern. This is a low-cost, proven technique that directly addresses the root cause. Option A over-engineers the solution with added latency and complexity. Option C misdiagnoses the problem—tool selection isn't the issue since single-concern accuracy is high. Option D is reactive rather than preventive and doesn't address the parameter mixing issue.

Option-level rationale: - A) Using a separate model call to decompose multi-concern messages adds unnecessary latency, complexity, and cost when the agent already demonstrates strong single-concern understanding. This over-engineers the solution when simpler prompt-level guidance can address the pattern recognition gap. - B) Adding few-shot examples demonstrating correct reasoning and tool sequencing for multi-concern requests is the most effective approach because the agent already handles individual concerns well at 94% accuracy—it simply needs pattern guidance for handling multiple concerns in one message. This is a low-cost, proven technique that directly addresses the root cause of the agent failing to decompose and properly route parameters across multiple requests. - C) Consolidating tools into fewer general-purpose tools misdiagnoses the problem, since the agent's high 94% accuracy on single-concern requests shows that tool selection and design are not the issue. The problem lies in the agent's inability to recognize and separately handle multiple concerns, not in having too many tools. - D) Detecting incomplete responses and re-prompting is a reactive strategy that only addresses missed concerns after the fact and

does not prevent the core issue of parameter mixing between requests. A preventive approach that teaches the agent to correctly handle multi-concern messages from the start is more effective and efficient.

Question 14

Topic: Batch Processing | Difficulty: easy | Scenario: 16 | ID: 657

Your CI/CD system performs three types of Claude-powered analysis: (1) quick style checks on each PR that block merging until complete, (2) comprehensive security audits of the entire codebase run weekly, and (3) test case generation triggered nightly for recently-modified modules. The Message Batches API offers 50% cost savings but can take up to 24 hours to process. You want to optimize API costs while maintaining acceptable developer experience. Which combination correctly matches each task to its API approach?

- **A)** Use synchronous calls for PR style checks; use the Message Batches API for weekly security audits and nightly test generation.
- **B)** Use the Message Batches API for all three tasks to maximize the 50% cost savings, and configure the pipeline to poll for batch completion.
- **C)** Use synchronous calls for PR style checks and nightly test generation; use Message Batches API only for weekly security audits.
- **D)** Use synchronous calls for all three tasks for consistent response times, and rely on prompt caching to reduce costs across all workloads.

Correct Answer: A - Use synchronous calls for PR style checks; use the Message Batches API for weekly security audits and nightly test generation.

Explanation: The correct answer matches API approaches to latency requirements: PR style checks block developers and need immediate responses (synchronous), while weekly audits and nightly test generation are scheduled tasks that can tolerate up to 24 hours of processing time (batch). Option B would create unacceptable delays for blocking PR checks. Option C misses cost savings on nightly jobs that don't need immediate results. Option D foregoes significant batch savings when prompt caching isn't an equivalent cost reduction.

Option-level rationale: - A) This is the correct approach. PR style checks block developers and require immediate responses via synchronous calls, while weekly security audits and nightly test generation are scheduled tasks with flexible timelines that can easily tolerate the up-to-24-hour batch processing window, capturing the 50% cost savings on both. - B) Using the Message Batches API for all three tasks would create unacceptable delays for PR style checks, which block merging and require immediate feedback for developers. While this maximizes cost savings, the up-to-24-hour processing time makes it unsuitable for latency-sensitive, developer-blocking workflows. - C) While correctly using synchronous calls for latency-sensitive PR checks, this approach misses the 50% cost savings on nightly test generation, which runs on a scheduled basis and can easily tolerate batch processing times. There is no reason to pay full price for a task that doesn't need immediate results. - D) Using synchronous calls for all tasks foregoes the significant 50% cost savings available through the Message Batches API for scheduled, non-latency-sensitive workloads. Prompt caching, while useful, does not provide an equivalent level of cost reduction and is not a substitute for batch processing savings on appropriate tasks.

Question 15

Topic: Escalation Decisions | Difficulty: hard | Scenario: 1 | ID: 666

After calling `get_customer` and `lookup_order`, the agent has retrieved all available system data but faces uncertainty. Which situation represents the most appropriate trigger for calling `escalate_to_human`?

- **A)** The customer's message mentions both a billing question and a product return. The agent should escalate so a human can coordinate handling both issues in a single interaction.
- **B)** The customer requests a price match against a competitor. Your policies allow adjustments for price drops on your own site within 14 days but are silent on competitor pricing. The agent should escalate for policy interpretation.
- **C)** The customer wants to cancel an order that shipped yesterday, with delivery scheduled for tomorrow. The agent should escalate because the customer might change their mind once they receive the package.

- **D)** The customer claims they never received their order, but tracking shows it was delivered and signed for at their address three days ago. The agent should escalate because presenting contradictory evidence might damage the customer relationship.

Correct Answer: B - The customer requests a price match against a competitor. Your policies allow adjustments for price drops on your own site within 14 days but are silent on competitor pricing. The agent should escalate for policy interpretation.

Explanation: Option B is correct because it identifies a genuine policy gap—the policy explicitly covers own-site price drops but is silent on competitor price matching. The agent cannot invent policy, so human judgment is required. Option A is wrong because multi-issue requests can be handled sequentially without escalation. Option C is wrong because escalating based on speculation about future customer intent is not a valid criterion. Option D is wrong because the stated reason (avoiding relationship damage) reflects emotional avoidance rather than operational necessity—the agent should present factual tracking evidence according to policy.

Option-level rationale: - A) Handling multiple issues such as a billing question and a product return does not require escalation, as the agent can address each topic sequentially within the same interaction using available tools and policies. - B) This represents a genuine policy gap where the company's guidelines cover own-site price drops but are silent on competitor price matching, meaning the agent cannot fabricate a policy and must escalate for human judgment on how to interpret or extend existing rules. - C) Escalating based on speculation that the customer might change their mind after receiving the package is not a valid trigger; the agent should address the customer's current, clearly stated request rather than deferring action based on hypothetical future intent. - D) While the situation involves contradictory information, the agent has factual tracking data to share with the customer per standard procedure; escalating to avoid presenting evidence out of concern for relationship damage reflects emotional avoidance rather than an operational need for human intervention.

Question 16

Topic: Batch Processing | Difficulty: hard | Scenario: 16 | ID: 681

The code review component works iteratively: Claude analyzes a changed file, then may request related files (imports, base classes, tests) via tool calling to understand context before providing final feedback. Your application defines a tool that lets Claude request file contents; Claude invokes this tool, receives results, and continues its analysis. You're evaluating batch processing to reduce API costs.

What is the primary technical constraint when considering batch processing for this workflow?

- **A)** The batch API doesn't support tool definitions in request parameters.
- **B)** The asynchronous model prevents executing tools mid-request and returning results for Claude to continue analysis.
- **C)** Batch processing lacks request correlation identifiers for matching outputs to input requests.
- **D)** Batch processing latency of up to 24 hours is too slow for pull request feedback, though the workflow could otherwise function.

Correct Answer: B - The asynchronous model prevents executing tools mid-request and returning results for Claude to continue analysis.

Explanation: Batch processing supports tool definitions and can generate tool calls, but its asynchronous fire-and-forget model means you cannot execute tools and return results mid-request. This breaks iterative workflows where Claude needs tool results to continue analysis. Option A is incorrect (batch fully supports tools), C is incorrect (custom_id provides request correlation), and D identifies a real but secondary concern—the phrase “though the workflow could otherwise function” is false because the architectural limitation in B would prevent this workflow even if batch completed instantly.

Option-level rationale: - A) The batch API fully supports tool definitions in request parameters and can generate tool use responses. This is not a constraint of batch processing. - B) This is correct. The batch API's asynchronous fire-and-forget model means there is no mechanism to intercept a tool call mid-request, execute the tool, and return results for Claude to continue its analysis. This fundamentally breaks iterative tool-calling workflows that require multiple rounds of tool invocation and response within a single logical interaction. - C) Batch processing provides a custom_id field for each request, which serves as a correlation identifier for matching outputs back to their corresponding inputs. This is not a limitation of the batch API. - D) While the up-to-24-hour latency is a real practical concern, the claim that the workflow could otherwise

function is incorrect. The fundamental architectural limitation is that batch processing cannot support the iterative tool-calling loop this workflow requires, regardless of how fast results are returned.

Question 17

Topic: Path-Specific Rule Configuration | Difficulty: medium | Scenario: 11 | ID: 682

Your codebase has distinct areas with different coding conventions: React components use functional style with hooks, API handlers use `async/await` with specific error handling, and database models follow a repository pattern. Test files are spread throughout the codebase alongside the code they test (e.g., `Button.test.tsx` next to `Button.tsx`), and you want all tests to follow the same conventions regardless of location. What's the most maintainable way to ensure Claude automatically applies the correct conventions when generating code?

- **A)** Create rule files in `.claude/rules/` with YAML frontmatter specifying glob patterns to conditionally apply conventions based on file paths
- **B)** Consolidate all conventions in the root `CLAUDE.md` file under headers for each area, relying on Claude to infer which section applies
- **C)** Create skills in `.claude/skills/` for each code type that include the relevant conventions in their `SKILL.md` files
- **D)** Place a separate `CLAUDE.md` file in each subdirectory containing that area's specific conventions

Correct Answer: A - Create rule files in `.claude/rules/` with YAML frontmatter specifying glob patterns to conditionally apply conventions based on file paths

Explanation: Option A is correct because `.claude/rules/` with glob patterns (e.g., `**/*.test.tsx`) allows conventions to be automatically applied based on file paths regardless of directory location—essential for test files spread throughout the codebase. Option B relies on inference rather than explicit matching, making it unreliable. Option C requires manual skill invocation or relies on Claude choosing to load them, contradicting the need for deterministic “automatic” application based on file paths. Option D can't

easily handle files spread across many directories since CLAUDE.md files are directory-bound.

Option-level rationale: - A) Using rule files in `.claude/rules/` with YAML frontmatter and glob patterns (e.g., `**/*.test.tsx`, `src/api/**/*.ts`) allows conventions to be automatically and deterministically applied based on file paths, regardless of where those files are located in the directory structure. This is the most maintainable approach because it handles cross-cutting concerns like test files spread throughout the codebase without requiring duplication or manual intervention. - B) Consolidating all conventions in a single root CLAUDE.md and relying on Claude to infer which section applies is unreliable because there is no deterministic mechanism ensuring the correct conventions are matched to the correct files. This approach may lead to inconsistent application of conventions, especially for files like tests that exist alongside different code types throughout the codebase. - C) Skills in `.claude/skills/` are designed for task-based workflows and typically require manual invocation or rely on Claude choosing to load them, which contradicts the requirement for automatic, deterministic application of conventions based on file paths. This approach lacks the glob-pattern-based conditional triggering needed to reliably match conventions to specific file types. - D) Placing separate CLAUDE.md files in each subdirectory works for directory-scoped conventions but cannot efficiently handle test files that are co-located with source files across many directories, since you would need to duplicate test conventions in every directory containing tests. This approach becomes unmaintainable as the codebase grows and doesn't support cross-cutting concerns that span multiple directories.

Question 18

Topic: Subagent Delegation Strategy | Difficulty: medium | Scenario: 11 | ID: 690

You're adding error handling wrappers to external API calls across a 120-file codebase. The task has three phases: (1) discovering all API call locations and patterns, (2) designing the error handling approach collaboratively, and (3) implementing wrappers consistently. During Phase 1, Claude generates verbose output listing hundreds of call sites with context. Your context window is filling rapidly before you've finished discovery.

What's the most effective approach to complete this while maintaining implementation consistency?

- **A)** Use the Explore subagent for Phase 1 to isolate verbose output and return a summary, then continue Phases 2-3 in the main conversation.
- **B)** Continue all phases in the main conversation, using /compact periodically to reduce context usage as you progress through the files.
- **C)** Define your error handling pattern in CLAUDE.md, then process files in batches across multiple sessions, relying on the shared memory file for consistency.
- **D)** Switch to headless mode with –continue, passing explicit context summaries between batch invocations to maintain continuity.

Correct Answer: A - Use the Explore subagent for Phase 1 to isolate verbose output and return a summary, then continue Phases 2-3 in the main conversation.

Explanation: The Explore subagent is designed for tasks producing verbose output you don't need retained, returning concise summaries. Keeping Phases 2 and 3 in the main conversation preserves context for iterative refinement and multi-phase work. Option B's /compact is lossy and may discard needed pattern details. Option C loses session context about edge case decisions across files. Option D adds manual overhead that subagents handle automatically and is designed for automation, not interactive work.

Option-level rationale: - A) Using the Explore subagent for Phase 1 is ideal because it isolates the verbose discovery output in a separate context, returning only a concise summary to the main conversation. This preserves the main context window for the collaborative design and consistent implementation phases where retained context is most valuable. - B) While /compact can reclaim context space, it is a lossy compression that may discard important pattern details and edge cases discovered during Phase 1, undermining the consistency needed for Phases 2 and 3. - C) Splitting work across multiple sessions loses the in-session context about nuanced decisions and edge cases encountered during discovery, and CLAUDE.md alone is insufficient to capture all the detailed rationale needed for consistent implementation across 120 files. - D) Headless mode with –continue is designed for automation rather than interactive collaborative work, and manually passing context summaries between invocations adds significant overhead that subagents handle automatically and more reliably.

Question 19

Topic: Context Provision Methods | Difficulty: easy | Scenario: 16 | ID: 693

Your automated review generates test case suggestions for each PR. When reviewing a PR that adds course completion tracking, Claude suggests 10 test cases but developer feedback indicates 6 duplicate scenarios already covered in the existing test suite. What change would most effectively reduce duplicate suggestions?

- **A)** Include the existing test file in the context so Claude can identify what scenarios are already covered
- **B)** Add instructions directing Claude to focus exclusively on edge cases and error conditions rather than successful paths
- **C)** Implement post-processing that filters suggestions whose descriptions match keywords from existing test names
- **D)** Reduce requested suggestions from 10 to 5, assuming Claude will prioritize the most valuable cases first

Correct Answer: A - Include the existing test file in the context so Claude can identify what scenarios are already covered

Explanation: The correct answer is A because LLMs can only reason about information in their context. Without knowing what tests already exist, Claude cannot avoid suggesting duplicates. Providing the existing test file directly addresses this root cause. Option B doesn't solve duplication since edge cases might also be covered. Option C uses brittle keyword matching that misses semantically equivalent tests with different names. Option D incorrectly assumes reducing quantity improves relevance—Claude has no basis to prioritize non-duplicates without context about what exists.

Option-level rationale: - A) Including the existing test file in the context directly addresses the root cause of duplication: Claude can only avoid suggesting already-covered scenarios if it knows what tests already exist. This gives Claude the information needed to reason about which suggestions would be genuinely new and valuable. - B) Restricting suggestions to only edge cases and error conditions does not solve the duplication problem, since existing tests may already cover those edge cases and error conditions. Without knowledge of the existing test suite, Claude would still have no way to avoid suggesting scenarios that are already tested. - C) Using keyword matching to filter suggestions is a brittle approach that would miss semantically equivalent tests

described with different wording or terminology. This post-processing workaround addresses symptoms rather than the root cause and would be unreliable in practice. - D) Simply reducing the number of requested suggestions does not give Claude any information about which scenarios are already covered, so it has no basis to prioritize non-duplicate cases. This approach incorrectly assumes that fewer suggestions will naturally be more unique, when in reality the most obvious suggestions are likely the ones already in the test suite.

Question 20

Topic: Agentic Loop Fundamentals | Difficulty: easy | Scenario: 1 | ID: 701

You're implementing the agentic loop for your support agent. After each API call to Claude, you need to determine whether to continue the loop (execute the requested tools and call Claude again) or stop (present the final response to the customer). What determines this decision?

- **A)** Check the `stop_reason` field in Claude's response—continue when it equals `"tool_use"` and stop when it equals `"end_turn"`.
- **B)** Set a maximum iteration count (e.g., 10 calls) and stop when reached, regardless of whether Claude indicates more work is needed.
- **C)** Parse Claude's response text for phrases like "I've completed" or "Is there anything else?"—these natural language signals indicate the task is finished.
- **D)** Check whether the response includes any assistant text content—if Claude generated explanatory text, the loop should end.

Correct Answer: A - Check the `stop_reason` field in Claude's response—continue when it equals `"tool_use"` and stop when it equals `"end_turn"`.

Explanation: The `stop_reason` field is Claude's explicit signal for loop control: `"tool_use"` means Claude wants to execute a tool and receive results, while `"end_turn"` means Claude has finished its response. Option B describes a safety guardrail, not the primary control mechanism. Option C relies on unreliable natural language parsing instead of structured API signals. Option D is incorrect because Claude often generates

both text and tool calls in the same response—text presence doesn't indicate completion.

Option-level rationale: - A) This is correct. The `stop_reason` field is Claude's explicit, structured signal for loop control: `"tool_use"` indicates Claude wants to execute a tool and receive the results back, while `"end_turn"` indicates Claude has completed its response and the loop should terminate. - B) While setting a maximum iteration count is a useful safety guardrail to prevent runaway loops, it is not the primary mechanism for determining whether to continue or stop the agentic loop. The loop should be driven by Claude's explicit signals about whether it needs to use more tools, not by an arbitrary iteration limit. - C) Parsing natural language phrases to detect completion is unreliable and fragile, as Claude may use varied phrasing or include such phrases mid-task. The API provides a structured `stop_reason` field specifically designed for this purpose, making text parsing unnecessary and error-prone. - D) This approach is incorrect because Claude frequently generates explanatory text alongside tool use requests in the same response. The presence of assistant text does not indicate that the task is complete, since Claude may be explaining its reasoning while simultaneously requesting a tool call.

Question 21

Topic: Tool Selection Reliability | Difficulty: easy | Scenario: 1 | ID: 703

In testing, you notice the agent frequently calls `get_customer` when users ask about order status, even though `lookup_order` would be more appropriate. What should you examine first to address this issue?

- **A)** Review tool descriptions to ensure they clearly distinguish each tool's purpose
- **B)** Add few-shot examples covering every possible order-related query pattern to the system prompt
- **C)** Implement a pre-processing classifier that detects order queries and routes directly to `lookup_order`
- **D)** Reduce the number of tools available to the agent to simplify selection

Correct Answer: A - Review tool descriptions to ensure they clearly distinguish each tool's purpose

Explanation: Tool descriptions are the primary input the model uses to decide which tool to call. When an agent consistently selects the wrong tool, the first diagnostic step is always to examine whether tool descriptions clearly distinguish their purposes and specify when each should be used. Option B's "every possible pattern" approach is impractical and addresses symptoms rather than root cause. Option C over-engineers the solution by building a separate classifier instead of fixing the underlying tool definitions. Option D sacrifices necessary functionality rather than addressing why the confusion exists.

Option-level rationale: - A) Tool descriptions are the primary input the model uses to decide which tool to call. When an agent consistently selects the wrong tool, the first diagnostic step is to examine whether the tool descriptions clearly distinguish each tool's purpose and specify when each should be used. - B) Attempting to add few-shot examples covering every possible order-related query pattern is impractical and addresses symptoms rather than the root cause, which is likely ambiguous or overlapping tool descriptions. - C) Building a separate pre-processing classifier to route queries is an over-engineered solution that bypasses the agent's native tool selection ability instead of fixing the underlying tool definitions that guide that selection. - D) Reducing the number of available tools sacrifices necessary functionality rather than addressing the root cause of why the agent confuses the two tools, which is most likely unclear tool descriptions.

Question 22

Topic: Custom Slash Commands | Difficulty: medium | Scenario: 11 | ID: 706

Your team created an `/analyze-codebase` skill that performs comprehensive code analysis—dependency scanning, test coverage calculation, and code quality metrics. After running this command, team members report that Claude becomes less responsive in the session and loses track of their original task. What's the most effective way to address this while preserving full analysis capability?

- **A)** Add `context: fork` to the skill's frontmatter to run the analysis in an isolated sub-agent context
- **B)** Add instructions to the skill to compress all outputs into a brief summary before displaying
- **C)** Split the skill into three smaller skills that each generate less output
- **D)** Add `model: haiku` to the frontmatter to use a faster, more efficient model for the analysis

Correct Answer: A - Add `context: fork` to the skill's frontmatter to run the analysis in an isolated sub-agent context

Explanation: The `context: fork` frontmatter option runs skills in an isolated sub-agent context, preventing verbose output from polluting the main conversation and causing responsiveness issues. Option B violates the requirement to preserve full analysis capability by compressing outputs. Option C doesn't solve the problem—running three skills sequentially still pollutes context with the same total output. Option D misunderstands the issue; model tier affects speed and cost but doesn't address context window pollution from analysis output. Note: Custom slash commands have been merged into skills in Claude Code. Files at `.claude/commands/` still work, but `.claude/skills/` is now the recommended location with additional features like supporting files and automatic loading.

Option-level rationale: - A) Using `context: fork` in the skill's frontmatter runs the analysis in an isolated sub-agent context, which prevents the verbose output from polluting the main conversation's context window and causing Claude to lose track of the original task. This preserves full analysis capability while keeping the main session responsive. - B) Compressing all outputs into a brief summary would sacrifice the full analysis capability that the question requires to be preserved. While it might reduce context pollution, it fundamentally undermines the purpose of comprehensive code analysis by discarding detailed results. - C) Splitting into three smaller skills doesn't solve the core problem, because running them sequentially still produces the same total volume of output in the main conversation context. The combined output from three skills would pollute the context window just as much as one large skill. - D) Switching to a faster model addresses speed and cost but does not solve the actual issue, which is context window pollution from verbose analysis output causing Claude to lose track of the user's original task. The responsiveness problem stems from context overload, not model performance.

Question 23

Topic: Custom Slash Commands | Difficulty: medium | Scenario: 11 | ID: 709

You've found that including 2-3 full exemplar endpoint implementations as context significantly improves consistency when generating new API endpoints. However, this context is only useful for creating new endpoints—not for bug fixes, code reviews, or other API directory work. What's the most efficient configuration approach?

- **A)** Add the exemplar endpoint code with pattern documentation to the project CLAUDE.md file so it's automatically available.
- **B)** Create a skill that references the exemplar endpoints and includes pattern-following instructions, invoked on-demand via slash command.
- **C)** Configure path-specific rules in `.claude/rules/api/` that include the exemplar code and activate when working in the API directory.
- **D)** Reference the exemplar endpoints manually in each generation request by copying relevant code into your prompt.

Correct Answer: B - Create a skill that references the exemplar endpoints and includes pattern-following instructions, invoked on-demand via slash command.

Explanation: Skills provide on-demand context loading via slash commands (e.g., a skill at `.claude/skills/api-patterns/SKILL.md` creates `/api-patterns`), adding overhead only when explicitly invoked for endpoint generation. Option A loads the exemplar code for every session regardless of task type, wasting context on unrelated work. Option C uses location-based activation (path rules), which would still load the context for bug fixes and reviews in the API directory—exactly what the scenario says is unnecessary. Option D works but is manual and tedious, not leveraging Claude Code's configuration capabilities.

Option-level rationale: - A) Adding exemplar endpoint code to the project CLAUDE.md file would load this context for every session regardless of the task type, wasting context window on bug fixes, code reviews, and other work where it isn't needed. - B) Creating a skill with the exemplar endpoints and pattern-following instructions allows on-demand invocation via a slash command, ensuring the context is loaded only when generating new endpoints and not during unrelated tasks like bug fixes or code reviews.

- C) Path-specific rules that activate when working in the API directory would trigger for all API directory work, including bug fixes and code reviews—exactly the scenarios where the exemplar context is unnecessary and wasteful. - D) Manually copying exemplar endpoint code into each prompt would technically work, but it is tedious, error-prone, and fails to leverage Claude Code's built-in configuration capabilities for reusable context loading.

Question 24

Topic: CLAUDE.md Configuration Hierarchy | Difficulty: medium | Scenario: 11 | ID: 710

Your team has been using Claude Code for several months. Recently, three developers report that Claude correctly follows your “always include comprehensive error handling” guideline, but a fourth developer who just joined reports Claude isn’t following this guideline. All four developers are working in the same repository and have the latest code pulled. What’s the most likely cause and appropriate fix?

- **A)** The new developer’s `~/claude/CLAUDE.md` contains conflicting instructions that override the project settings. Have them remove the conflicting section from their user-level configuration.
- **B)** The guideline exists in the original developers’ `~/claude/CLAUDE.md` files (user-level) instead of the project’s `.claude/CLAUDE.md`. Move the instruction to the project-level file so all team members receive it.
- **C)** Claude Code builds per-user preference models over time through repeated interactions. The new developer needs to repeatedly specify the error handling requirement until Claude learns their preferences.
- **D)** Claude Code caches `CLAUDE.md` contents after first read. The original developers have cached versions while the new developer loaded after the file was modified. Have all developers clear their Claude Code cache.

Correct Answer: B - The guideline exists in the original developers’ `~/claude/CLAUDE.md` files (user-level) instead of the project’s `.claude/CLAUDE.md`. Move the instruction to the project-level file so all team members receive it.

Explanation: B is correct because if team guidelines were added to individual developers’ user-level `~/claude/CLAUDE.md` files instead of the project’s

.claude/CLAUDE.md, only those developers would receive the instructions. Moving guidelines to the project-level file ensures all team members (including new joiners) receive them. A is unlikely since new developers rarely have pre-existing conflicting configurations. C is incorrect because Claude Code doesn't build persistent per-user preference models through repeated interactions. D is incorrect because Claude Code reads CLAUDE.md files fresh at session start rather than caching them across sessions.

Option-level rationale: - A) While conflicting user-level instructions could theoretically cause issues, it is unlikely that a newly joined developer would already have pre-existing conflicting configurations in their user-level CLAUDE.md file. This scenario doesn't explain why the three existing developers all have the guideline working consistently. - B) This is the most likely cause: if the error handling guideline was added to each original developer's user-level ~/.claude/CLAUDE.md rather than the project's .claude/CLAUDE.md, new team members would not receive it. Moving the instruction to the project-level configuration file ensures all current and future team members automatically receive the guideline. - C) Claude Code does not build persistent per-user preference models through repeated interactions across sessions. Each session starts fresh using the instructions from CLAUDE.md files and the current conversation context, so repeatedly specifying a requirement would not cause Claude to "learn" it permanently. - D) Claude Code reads CLAUDE.md files fresh at the start of each session rather than caching them across sessions. There is no cross-session caching mechanism that would cause different developers to see different versions of the same configuration file.

Question 25

Topic: Multi-step Workflow Orchestration | Difficulty: medium | Scenario: 1 | ID: 712

Production logs show that for simple requests like "refund order #1234", your agent succeeds in 3-4 tool calls with 91% resolution rate. However, for complex requests like "I've been charged twice, my discount didn't apply, and I want to cancel", the agent averages 12+ tool calls with only 54% resolution—often investigating concerns sequentially and gathering redundant customer data for each one. What's the most effective change to improve complex request handling?

- **A)** Add explicit verification gates between steps requiring the agent to checkpoint after resolving each concern before moving to the next.
- **B)** Decompose the request into distinct concerns, then investigate each in parallel using shared customer context before synthesizing a resolution.
- **C)** Add few-shot examples demonstrating ideal tool call sequences for various multi-part billing scenarios to your system prompt.
- **D)** Reduce the number of available tools by consolidating `get_customer`, `lookup_order`, and billing-related lookups into a single `investigate_issue` tool.

Correct Answer: B - Decompose the request into distinct concerns, then investigate each in parallel using shared customer context before synthesizing a resolution.

Explanation: Option B (decompose-parallelize-synthesize) directly addresses both problems: sequential investigation causing excessive tool calls, and redundant customer data gathering. By decomposing the request into distinct concerns first and reusing the same customer context across parallel investigations, the agent eliminates redundancy and reduces total calls. Option A would make the sequential problem worse by adding more gates. Option C doesn't systematically address context reuse or parallel investigation. Option D hides useful tool structure and doesn't address the workflow pattern issue.

Option-level rationale: - A) Adding verification gates between sequential steps would actually worsen the problem by reinforcing the sequential processing pattern and adding overhead, rather than addressing the root cause of redundant data gathering and serial investigation. - B) Decomposing the request into distinct concerns and investigating them in parallel with shared customer context directly addresses both core issues: it eliminates redundant data fetching by reusing context across concerns and reduces total tool calls by parallelizing investigations before synthesizing a unified resolution. - C) While few-shot examples might marginally improve tool call sequences for known scenarios, they don't systematically solve the structural problems of redundant context gathering or sequential investigation, and they won't generalize well to novel multi-part request combinations. - D) Consolidating multiple tools into a single investigation tool obscures useful functional distinctions and doesn't address the underlying workflow pattern problem of sequential processing and redundant data gathering across concerns.

Question 26

Topic: CI/CD Integration | Difficulty: medium | Scenario: 16 | ID: 713

After an initial automated review generates 12 findings, a developer pushes new commits to address the issues. When the review runs again, it produces 8 findings—but developers report that 5 duplicate earlier comments on code that was already fixed in the new commits. What's the most effective way to eliminate this redundant feedback while maintaining thorough analysis?

- **A)** Include prior review findings in context, instructing Claude to only report new or still-unaddressed issues.
- **B)** Restrict the review scope to only files modified in the most recent push, excluding files from earlier commits.
- **C)** Add a post-processing filter that removes findings matching previous file paths and issue descriptions before posting comments.
- **D)** Run reviews only on initial PR creation and final pre-merge state, skipping intermediate commits.

Correct Answer: A - Include prior review findings in context, instructing Claude to only report new or still-unaddressed issues.

Explanation: Option A is correct because it provides Claude with the context needed to make intelligent decisions about what's genuinely new versus already addressed. By including previous findings, Claude can reason about whether issues have been fixed rather than blindly re-flagging them. Option B sacrifices thoroughness by limiting scope—changes in one file might affect issues in another. Option C uses brittle pattern matching that can't semantically determine if issues are truly resolved, and LLM outputs vary in wording across runs. Option D avoids the problem by reducing feedback frequency, losing the continuous feedback value that makes CI/CD reviews useful.

Option-level rationale: - A) Including prior review findings in context allows Claude to intelligently distinguish between new issues and those already addressed by recent commits. This approach maintains thorough analysis while leveraging Claude's reasoning ability to avoid redundant feedback on fixed code. - B) Restricting the review scope to only recently modified files sacrifices thoroughness, since changes in one file can introduce or affect issues in other files that weren't directly modified. This approach trades analysis quality for deduplication, which is not an ideal tradeoff. - C) A post-

processing filter based on matching file paths and issue descriptions is brittle because LLM-generated comments vary in wording across runs, making exact matching unreliable. It also cannot semantically determine whether an issue has actually been resolved or merely rephrased differently. - D) Running reviews only at PR creation and final pre-merge state eliminates the continuous feedback loop that makes CI/CD-integrated reviews valuable. This sidesteps the duplication problem rather than solving it, and developers lose the benefit of iterative review feedback during development.

Question 27

Topic: Custom Slash Commands | Difficulty: medium | Scenario: 11 | ID: 715

You're creating a custom `/explore-alternatives` skill that your team uses to brainstorm and evaluate different implementation approaches before committing to one. However, developers report that after running this skill, Claude's subsequent responses are influenced by the exploration discussion—sometimes referencing abandoned approaches or maintaining exploratory context that confuses actual implementation work. What's the most effective way to configure this skill?

- **A)** Add `context: fork` to the skill's frontmatter.
- **B)** Split the skill into two separate skills—`/explore-start` and `/explore-end`—to demarcate when exploration context should be discarded.
- **C)** Create the skill in `~/.claude/skills/` instead of `.claude/skills/`.
- **D)** Use the `!` prefix in the skill to execute the exploration logic as a bash subprocess.

Correct Answer: A - Add `context: fork` to the skill's frontmatter.

Explanation: The `context: fork` frontmatter option runs the skill in an isolated sub-agent context, preventing exploration discussion from affecting subsequent conversation. Option B fails because there's no mechanism to discard context mid-conversation—history is additive. Option C is wrong because skill directory location affects discoverability and precedence, not execution context. Option D misunderstands the `!` prefix—bash output still flows into conversation context, and exploration requires LLM reasoning.

Option-level rationale: - A) The `context: fork` frontmatter option runs the skill in an isolated sub-agent context, so the exploration discussion does not pollute the main conversation history. This prevents abandoned approaches and exploratory context from influencing subsequent implementation work. - B) Splitting the skill into start and end commands does not solve the problem because there is no mechanism to discard context mid-conversation—conversation history is additive, and creating two skills cannot retroactively remove prior discussion from the context window. - C) Placing the skill in the user-level `~/.claude/skills/` directory instead of the project-level `.claude/skills/` directory only affects discoverability and precedence of the skill, not how its execution context is managed or isolated from subsequent conversation. - D) Using the `!` prefix to execute exploration logic as a bash subprocess misunderstands the feature—bash output still flows back into the conversation context, and meaningful brainstorming and evaluation of implementation approaches requires LLM reasoning rather than shell commands.

Question 28

Topic: Parallel Tool Execution | Difficulty: medium | Scenario: 1 | ID: 718

Production metrics show your agent averages 4+ API round-trips per resolution. Analysis reveals Claude frequently requests `get_customer` and `lookup_order` in separate sequential turns even when both are needed upfront. What's the most effective way to reduce round-trips?

- **A)** Prompt Claude to batch tool requests per turn, and return all tool results together before the next API call.
- **B)** Implement speculative execution that automatically calls likely-needed tools alongside any requested tool, returning all results regardless of what was requested.
- **C)** Create composite tools like `get_customer_with_orders` that bundle common lookup combinations into single calls.
- **D)** Increase `max_tokens` to give Claude more space to plan ahead and naturally batch its tool requests.

Correct Answer: A - Prompt Claude to batch tool requests per turn, and return all tool results together before the next API call.

Explanation: Claude can request multiple tools in a single response, and executors should return all results together before the next API call. Prompting Claude to batch related information-gathering calls leverages this native capability with minimal architecture changes. Speculative execution (B) wastes resources on unneeded calls and introduces complexity. Composite tools (C) reduce flexibility and create maintenance burden without addressing the root cause. Increasing max_tokens (D) doesn't affect tool batching behavior—it only limits output length.

Option-level rationale: - A) Prompting Claude to batch related tool requests in a single turn, and returning all results together before the next API call, leverages Claude's native ability to request multiple tools simultaneously. This is the most effective approach because it directly addresses the sequential calling pattern with minimal architectural changes. - B) Speculatively executing tools that weren't explicitly requested wastes resources on unneeded calls, introduces unnecessary complexity, and may return irrelevant or confusing results that don't align with Claude's actual reasoning process. - C) Creating composite tools reduces flexibility and increases maintenance burden by requiring new bundled tools for every common combination, without addressing the root cause—which is that Claude already supports requesting multiple tools per turn and simply needs to be prompted to do so. - D) Increasing max_tokens only affects the maximum length of Claude's text output and does not influence whether Claude batches multiple tool requests in a single turn, so this change would have no meaningful impact on reducing round-trips.

Question 29

Topic: MCP Server Integration | Difficulty: hard | Scenario: 11 | ID: 719

Your team wants to add a GitHub MCP server to enable PR lookups and CI status checks through Claude Code. Each of the six developers has their own GitHub personal access token. You want consistent tooling across the team without committing credentials to version control. What's the most effective configuration approach?

- **A)** Add the server to a project-scoped `.mcp.json` with environment variable expansion (`${GITHUB_TOKEN}`) for authentication, and document the required environment variable in your project README.

- **B)** Have each developer configure the server in user scope with `claude mcp add --scope user`.
- **C)** Configure the server in project scope with a placeholder token value, then instruct developers to override it in their local scope configuration.
- **D)** Create an MCP server wrapper that reads tokens from a `.env` file and proxies requests to the GitHub API, then add this wrapper to your project `.mcp.json`.

Correct Answer: A - Add the server to a project-scoped `.mcp.json` with environment variable expansion (`${GITHUB_TOKEN}`) for authentication, and document the required environment variable in your project README.

Explanation: Option A is correct because project-scoped `.mcp.json` with environment variable expansion is the idiomatic pattern for sharing MCP configurations—it provides a single source of truth for the team while allowing each developer to supply their own credentials via environment variables. Option B fails the consistency requirement since user-scoped configurations across six developers will inevitably diverge. Option C is an anti-pattern that commits placeholder values and relies on a fragile override mechanism. Option D is over-engineered, building a custom wrapper when native environment variable expansion already solves the problem.

Option-level rationale: - A) Using a project-scoped `.mcp.json` with environment variable expansion (`${GITHUB_TOKEN}`) is the idiomatic approach—it provides a single, version-controlled source of truth for the team’s MCP configuration while allowing each developer to supply their own credentials through environment variables. Documenting the required variable in the README ensures easy onboarding without ever committing secrets. - B) Having each developer independently configure the server in user scope fails the consistency requirement, as six separate configurations will inevitably diverge over time with no shared source of truth. This approach also increases onboarding friction since new developers must manually replicate the correct setup. - C) Committing a placeholder token value to the project-scoped configuration is an anti-pattern that could be mistaken for a real credential and relies on a fragile override mechanism that developers may forget or misconfigure. This approach adds unnecessary complexity compared to native environment variable expansion. - D) Building a custom wrapper to proxy GitHub API requests and read tokens from a `.env` file is over-engineered, since Claude Code’s native environment variable expansion in `.mcp.json` already solves the credential injection problem cleanly. This approach introduces additional maintenance burden and potential points of failure without meaningful benefit.

Question 30

Topic: Plan Mode vs Direct Execution | Difficulty: easy | Scenario: 11 | ID: 720

You need to add Slack as a new notification channel. The existing codebase has clear, consistent patterns for email, SMS, and push channels. However, the Slack API offers fundamentally different integration approaches—incoming webhooks (simple, one-way only), bot tokens (enables delivery confirmation and programmatic control), or Slack Apps (bidirectional events, requires workspace approval). Your ticket says “add Slack support” without specifying which integration method or whether advanced features like delivery tracking will be needed. How should you approach this task?

- **A)** Enter plan mode to explore the integration options and their architectural implications, then present a recommendation before implementing.
- **B)** Start direct execution using incoming webhooks to match the existing one-way notification pattern.
- **C)** Start direct execution using the bot token approach to enable delivery confirmation capabilities.
- **D)** Start direct execution to scaffold the Slack channel class following existing patterns, deferring the integration method decision until later.

Correct Answer: A - Enter plan mode to explore the integration options and their architectural implications, then present a recommendation before implementing.

Explanation: When facing multiple valid implementation approaches with different architectural implications AND ambiguous requirements, plan mode is appropriate. The Slack integration method choice (webhooks vs bot tokens vs Apps) affects authentication flows, error handling, and future capabilities—this architectural decision warrants exploration and team alignment before implementation. Options B and C both make assumptions about which approach is correct without evaluating trade-offs. Option D incorrectly assumes scaffolding can proceed meaningfully before the integration method is chosen, when the authentication pattern and response handling differ significantly between approaches.

Option-level rationale: - A) This is correct because the Slack integration involves multiple valid approaches with significantly different architectural implications, and the

requirements are ambiguous. Using plan mode to explore trade-offs between webhooks, bot tokens, and Slack Apps allows for an informed recommendation and team alignment before committing to an implementation path. - B) While incoming webhooks match the existing one-way notification pattern, this approach prematurely commits to a specific integration method without evaluating whether advanced features like delivery tracking might be needed. The ambiguous requirements warrant exploring all options before making an architectural decision. - C) Although bot tokens enable delivery confirmation, jumping directly to this approach assumes that delivery tracking is a requirement when the ticket doesn't specify this. Choosing a more complex integration method without evaluating trade-offs or clarifying requirements risks over-engineering the solution. - D) This approach incorrectly assumes that meaningful scaffolding can proceed before choosing the integration method, when in reality the authentication flows, response handling, and error patterns differ significantly between webhooks, bot tokens, and Slack Apps. Deferring the integration method decision would likely result in rework once the approach is selected.

Question 31

Topic: Tool Selection Reliability | Difficulty: medium | Scenario: 1 | ID: 721

Production logs reveal a consistent pattern: when customers include "account" in messages (e.g., "I want to check my account for the order I placed yesterday"), the agent calls `get_customer` first 78% of the time. When customers phrase similar requests without "account" (e.g., "I want to check on the order I placed yesterday"), it calls `lookup_order` first 93% of the time. The tool descriptions are well-written and unambiguous. What is the most likely root cause of this discrepancy?

- **A)** The system prompt contains keyword-sensitive instructions that steer behavior based on terms like "account," creating unintended tool selection patterns
- **B)** The tool descriptions need additional negative examples specifying when NOT to use each tool to prevent this keyword-triggered confusion
- **C)** The model requires more training data on multi-concept messages and should be fine-tuned on examples that include both account and order language
- **D)** The model's base training creates associations between "account" terminology and customer-related operations that override the tool descriptions

Correct Answer: A - The system prompt contains keyword-sensitive instructions that steer behavior based on terms like "account," creating unintended tool selection patterns

Explanation: The systematic pattern (78% vs 93%) triggered by a specific keyword indicates explicit routing logic in the system prompt rather than tool description issues or model limitations. Option B contradicts the stated premise that descriptions are unambiguous. Option C proposes impractical fine-tuning when simpler prompt fixes would address the issue. Option D is unlikely because the model correctly selects tools 93% of the time when the keyword is absent, demonstrating it can make correct selections without the triggering term.

Option-level rationale: - A) This is the most likely root cause because the systematic, keyword-triggered pattern (78% vs 93%) strongly suggests explicit routing logic in the system prompt that reacts to the word "account" and directs the agent toward customer-related tools. Since the tool descriptions are already well-written and unambiguous, the discrepancy points to prompt-level instructions creating unintended behavioral steering. - B) Adding negative examples to tool descriptions contradicts the stated premise that the descriptions are already well-written and unambiguous. The issue is not with the tool descriptions themselves but with upstream instructions in the system prompt that override correct tool selection based on keyword triggers. - C) Proposing fine-tuning is an impractical and disproportionate response when the systematic keyword-triggered pattern strongly suggests a simpler prompt-level fix would resolve the issue. The model already demonstrates correct tool selection 93% of the time when the triggering keyword is absent, indicating it does not need additional training data. - D) Base model associations are unlikely to be the root cause because the model correctly selects the appropriate tool 93% of the time when the word "account" is absent, demonstrating it can properly interpret the user's intent without interference. The sharp, keyword-dependent shift in behavior points to explicit prompt instructions rather than inherent model biases.

Question 32

Topic: Iterative Refinement | Difficulty: medium | Scenario: 11 | ID: 723

You've asked Claude Code to implement a function that transforms API responses into a normalized internal format. After two iterations, the output structure still doesn't match expectations—some fields are nested differently and timestamps aren't formatted correctly. You've been describing the requirements in prose, but Claude seems to interpret them differently each time. What's the most effective approach for the next iteration?

- **A)** Provide 2-3 concrete input-output examples showing the expected transformation for representative API responses.
- **B)** Rewrite your requirements with greater technical precision, specifying exact field mappings, nesting rules, and timestamp format strings.
- **C)** Write a JSON schema defining the expected output structure and validate Claude's output against it after each iteration.
- **D)** Ask Claude to explain its current interpretation of the requirements so you can identify where understanding diverges.

Correct Answer: A - Provide 2-3 concrete input-output examples showing the expected transformation for representative API responses.

Explanation: Concrete input-output examples are the most effective approach because they directly address the core problem: prose descriptions being interpreted ambiguously. Examples bypass linguistic ambiguity by showing exactly what transformation is expected. Option B continues using prose, which has already failed twice. Option C validates output structure but doesn't help Claude understand the transformation logic. Option D is diagnostic but doesn't provide a concrete target for iteration—after identifying divergence, you'd still need to use examples or other methods to fix it.

Option-level rationale: - A) Providing concrete input-output examples is the most effective approach because it eliminates the ambiguity inherent in prose descriptions by showing Claude exactly what the expected transformation looks like. This directly addresses the root cause—misinterpretation of prose requirements—by giving unambiguous, concrete targets for field nesting and timestamp formatting. - B) While more precise prose descriptions might help marginally, this approach continues relying on the same communication method that has already failed twice. Prose, no matter how technically precise, is still subject to interpretation differences, whereas concrete examples would eliminate ambiguity entirely. - C) A JSON schema can validate the output structure but doesn't help Claude understand the actual transformation logic

needed to produce correct results. This approach addresses verification rather than comprehension, meaning Claude would still need to understand the mapping requirements to generate correct output in the first place. - D) Asking Claude to explain its interpretation is a useful diagnostic step that can reveal where understanding diverges, but it doesn't directly solve the problem. After identifying the misunderstanding, you would still need another method—such as providing concrete examples—to communicate the correct transformation, making this an indirect and incomplete approach.

Question 33

Topic: CLAUDE.md Modular Organization | Difficulty: medium | Scenario: 11 | ID: 724

Your team's CLAUDE.md file has grown to over 500 lines, mixing TypeScript conventions, testing guidelines, API patterns, and deployment procedures. Developers find it difficult to locate and update relevant sections. What approach does Claude Code support for organizing project-level instructions into focused, topic-specific modules?

- **A)** Create separate markdown files in `.claude/rules/`, each covering one topic (e.g., `testing.md`, `api-conventions.md`)
- **B)** Create multiple files named `CLAUDE.md` at different levels of the directory tree, each one overriding the parent's instructions
- **C)** Define a `.claude/config.yaml` file that maps file patterns to specific sections within CLAUDE.md
- **D)** Split instructions into README.md files in relevant subdirectories, which Claude automatically loads as instructions

Correct Answer: A - Create separate markdown files in `.claude/rules/`, each covering one topic (e.g., `testing.md`, `api-conventions.md`)

Explanation: Claude Code supports a `.claude/rules/` directory where you can create separate markdown files for topic-specific guidelines (e.g., `testing.md`, `api-conventions.md`). This allows teams to organize large instruction sets into maintainable modules. Option B describes the directory hierarchy feature, which provides context for different parts of the codebase but doesn't solve the problem of organizing a single

project's many guidelines. Option C doesn't exist as a Claude Code feature. Option D confuses README files (human documentation) with CLAUDE.md files (instructions for Claude).

Option-level rationale: - A) This is correct. Claude Code supports a `.claude/rules/` directory where you can create separate markdown files for topic-specific guidelines (e.g., `testing.md`, `api-conventions.md`), allowing teams to organize large instruction sets into focused, maintainable modules. - B) While Claude Code does support placing CLAUDE.md files at different levels of the directory tree, this feature provides context scoped to different parts of the codebase rather than solving the problem of organizing a single project's many guidelines into topic-specific modules. Multiple CLAUDE.md files also do not override parent instructions; they supplement them. - C) A `.claude/config.yaml` file that maps file patterns to specific sections within CLAUDE.md does not exist as a Claude Code feature. There is no supported mechanism for conditionally loading sections of a CLAUDE.md file based on file pattern matching through a config file. - D) README.md files serve as human-facing documentation and are not automatically loaded by Claude Code as project instructions. Only CLAUDE.md files and files in the `.claude/rules/` directory are recognized as instruction sources for Claude.

Question 34

Topic: Custom Slash Commands | Difficulty: hard | Scenario: 11 | ID: 726

Your team has created a `/migration` skill that generates database migration files. The skill accepts a migration name via `$ARGUMENTS`. In production, you're seeing three issues: (1) developers often invoke the skill without arguments, resulting in poorly-named files, (2) the skill sometimes incorporates database schema details from unrelated earlier conversations, and (3) a developer accidentally triggered destructive test cleanup when the skill had broad tool access. Which configuration approach addresses all three issues?

- **A)** Add `argument-hint` frontmatter to prompt for required parameters, use `context: fork` to isolate execution, and restrict `allowed-tools` to file write operations.

- **B)** Split into separate `/migration-create` and `/migration-apply` skills, add instructions in each SKILL.md to request a migration name if not provided, and use different `allowed-tools` scopes for each skill.
- **C)** Include validation instructions in the skill's SKILL.md that direct Claude to verify `$ARGUMENTS` contains a valid name, add prompts to ignore prior conversation context, and list forbidden operations Claude should avoid.
- **D)** Use positional parameters `$1` and `$2` instead of `$ARGUMENTS` to enforce specific inputs, include explicit schema file references via `@` syntax to control context, and add `description` frontmatter warning about destructive operations.

Correct Answer: A - Add `argument-hint` frontmatter to prompt for required parameters, use `context: fork` to isolate execution, and restrict `allowed-tools` to file write operations.

Explanation: Option A correctly uses three distinct skill configuration features to address each specific issue: `argument-hint` frontmatter shows expected parameters during autocomplete (addressing missing arguments), `context: fork` isolates execution in a subagent context separate from conversation history (preventing context bleeding), and `allowed-tools` restricts tool access (preventing unauthorized operations). Option B attempts three fixes but splitting skills doesn't inherently solve context isolation, and relying on SKILL.md instructions to request arguments is less effective than `argument-hint` which provides autocomplete hints. Option C relies entirely on prompt-based instructions which are unreliable—skills are markdown templates that become part of the prompt, not programs with enforcement mechanisms; telling Claude to "ignore prior context" or "avoid forbidden operations" doesn't actually restrict access. Option D's positional parameters don't enforce argument presence (they're just empty if not provided), `@` file references control what's explicitly included but don't prevent context from prior conversation, and `description` frontmatter only appears in help text—it doesn't restrict tool access.

Option-level rationale: - A) This approach correctly uses three distinct skill configuration features to address each issue: `argument-hint` frontmatter shows expected parameters during autocomplete (addressing missing arguments), `context: fork` isolates execution in a subagent context separate from conversation history (preventing context bleeding from earlier conversations), and `allowed-tools` restricts tool access to only file write operations (preventing destructive actions). - B) Splitting into separate skills doesn't inherently solve the context isolation problem since both skills would still share conversation history without `context: fork`, and relying on

SKILL.md instructions to request a migration name is less reliable than using `argument-hint` frontmatter which provides autocomplete hints at invocation time. - C) This approach relies entirely on prompt-based instructions, which are unreliable for enforcement—telling Claude to “ignore prior context” doesn’t actually isolate execution context, and listing forbidden operations doesn’t restrict tool access the way `allowed-tools` does, leaving the system vulnerable to all three issues. - D) Positional parameters like `$1` and `$2` don’t enforce argument presence since they simply resolve to empty strings if not provided, `@` file references control what’s explicitly included but don’t prevent context bleeding from prior conversation turns, and `description` frontmatter only appears in help text without actually restricting tool access to prevent destructive operations.

Question 35

Topic: Multi-Agent Orchestration / Difficulty: easy / Scenario: 3 / ID: 731

A colleague suggests having the document analysis agent send its output directly to the synthesis agent instead of routing through the coordinator. What is the main advantage of keeping the coordinator as the central hub for all subagent communication?

- **A)** The coordinator can observe all interactions, handle errors consistently, and decide what information each subagent should receive
- **B)** Subagents operate with isolated memory, and direct communication would require complex serialization that only the coordinator can perform
- **C)** The coordinator batches multiple subagent requests together, reducing the total number of API calls and overall latency
- **D)** Routing through the coordinator enables automatic retry logic that direct agent-to-agent calls cannot support

Correct Answer: A - The coordinator can observe all interactions, handle errors consistently, and decide what information each subagent should receive

Explanation: The coordinator pattern provides centralized visibility, consistent error handling, and control over information flow between subagents. Option B is wrong because serialization isn’t unique to coordinators—any component can pass data.

Option C is incorrect because routing through a coordinator adds overhead rather than reducing it. Option D creates a false dichotomy—retry logic can be implemented anywhere, not just in coordinators.

Option-level rationale: - A) This is correct. The coordinator pattern provides centralized visibility into all interactions, consistent error handling across the system, and fine-grained control over what information each subagent receives, which are the primary advantages of hub-and-spoke communication. - B) This is incorrect. Serialization of data between components is not unique to coordinators—any component can serialize and pass data, so direct communication between subagents would not face a special serialization barrier that only a coordinator can overcome. - C) This is incorrect. Routing all communication through a coordinator actually adds overhead and an extra hop rather than reducing API calls or latency. Batching is not the primary rationale for the coordinator pattern. - D) This is incorrect. Automatic retry logic can be implemented at any level of the system, not exclusively within a coordinator. This creates a false dichotomy by implying that direct agent-to-agent calls inherently cannot support retries.

Question 36

Topic: Agent SDK Hook Patterns | Difficulty: medium | Scenario: 1 | ID: 732

Production logs reveal that the agent misinterprets data from your MCP tools: Unix timestamps from `get_customer`, ISO 8601 dates from `lookup_order`, and numeric status codes (1=pending, 2=shipped). Some tools are third-party MCP servers you cannot modify. What's the most maintainable approach to normalize data formats?

- **A)** Use a `PostToolUse` hook to intercept tool results and apply formatting transformations before agent processing
- **B)** Add detailed format documentation to your system prompt explaining each tool's data conventions
- **C)** Modify tools you control to return human-readable formats; create wrapper tools for third-party tools
- **D)** Create a `normalize_data` tool that the agent calls after each data retrieval to transform values

Correct Answer: A - Use a PostToolUse hook to intercept tool results and apply formatting transformations before agent processing

Explanation: PostToolUse hooks provide a single, centralized point for data normalization that works uniformly across all tools (including third-party ones you cannot modify). This approach is deterministic—code transforms data reliably, unlike option B which relies on probabilistic LLM interpretation. Option C creates inconsistent dual approaches (modify some tools, wrap others), violating maintainability principles. Option D requires the agent to remember to call a tool after every retrieval, doubling API calls and relying on unreliable agent behavior for deterministic transformations.

Option-level rationale: - A) Using a PostToolUse hook provides a centralized, deterministic point to intercept and normalize all tool outputs—including those from third-party MCP servers—before the agent processes them. This is the most maintainable approach because it applies transformations uniformly via code rather than relying on LLM interpretation or agent behavior. - B) Adding format documentation to the system prompt relies on the LLM to probabilistically interpret and convert data formats correctly, which is unreliable for deterministic transformations. This approach is also fragile, as the agent may still misinterpret formats despite the documentation, especially under complex reasoning scenarios. - C) Modifying tools you control while wrapping third-party tools creates two different normalization strategies, resulting in an inconsistent and harder-to-maintain architecture. This dual approach increases complexity and makes it difficult to ensure uniform data formatting across all tools. - D) Requiring the agent to call a separate normalization tool after every data retrieval doubles API calls and depends on the agent reliably remembering to invoke it each time. This approach is fragile because agents can forget or skip steps, making it unsuitable for deterministic data transformations.

Question 37

Topic: Skills vs CLAUDE.md Scope | Difficulty: medium | Scenario: 11 | ID: 734

Your CLAUDE.md has grown to over 400 lines containing coding standards, testing conventions, a detailed PR review checklist, deployment workflow instructions, and database migration procedures. You want Claude to always follow the coding standards and testing conventions, but only apply PR review, deployment, and migration guidance

when you're actually performing those tasks. What's the most effective restructuring approach?

- **A)** Move all guidance into separate Skills files organized by workflow type, keeping only a brief project description in CLAUDE.md
- **B)** Keep universal standards in CLAUDE.md and create Skills for task-specific workflows (PR reviews, deployments, migrations) with trigger keywords
- **C)** Split the CLAUDE.md into files in .claude/rules/ with path-specific glob patterns so each rule loads only for matching file types
- **D)** Keep all content in CLAUDE.md but use @import syntax to organize it into separately maintained files by category

Correct Answer: B - Keep universal standards in CLAUDE.md and create Skills for task-specific workflows (PR reviews, deployments, migrations) with trigger keywords

Explanation: CLAUDE.md content is loaded for every conversation, making it ideal for universal standards you always want applied. Skills are model-invoked and loaded on-demand when Claude detects a relevant task, making them perfect for specialized workflows. Option A fails because moving everything to Skills means coding standards won't be automatically applied. Option C misapplies path-specific rules (designed for file-type conventions) to workflow-based distinctions. Option D improves maintainability through @import but still loads all content every conversation, not solving the conditional loading requirement.

Option-level rationale: - A) Moving all guidance into Skills files means that universal coding standards and testing conventions won't be automatically loaded into every conversation. Since these standards should always be applied, they belong in CLAUDE.md where they are included by default. - B) This is the most effective approach because CLAUDE.md content is loaded for every conversation, ensuring coding standards and testing conventions are always applied, while Skills are invoked on-demand when Claude detects relevant trigger keywords, making them ideal for task-specific workflows like PR reviews, deployments, and migrations. - C) Path-specific glob patterns in .claude/rules/ are designed to apply rules based on file types being edited, not based on workflow tasks like PR reviews or deployments. This approach misapplies file-path matching to workflow-based distinctions that don't correspond to specific file types. - D) While using @import syntax to organize content into separate files improves maintainability, all imported content would still be loaded into every conversation. This

does not solve the core requirement of conditionally loading PR review, deployment, and migration guidance only when those tasks are being performed.

Question 38

Topic: Custom Slash Commands | Difficulty: hard | Scenario: 11 | ID: 736

You've created a `/commit` skill in `.claude/skills/commit/SKILL.md` that your team uses. One developer wants to customize it for their personal workflow (different commit message format, additional checks) without affecting teammates. What should you recommend?

- **A)** Create a personal version in `~/.claude/skills/` with a different name like `/my-commit`
- **B)** Create a personal version at `~/.claude/skills/commit/SKILL.md` with the same name
- **C)** Set `override: true` in the personal skill's frontmatter to take precedence over the project version
- **D)** Add username-based conditional logic to the project skill's frontmatter

Correct Answer: A - Create a personal version in `~/.claude/skills/` with a different name like `/my-commit`

Explanation: Project skills in `.claude/skills/` take precedence over personal skills in `~/.claude/skills/` when they share the same name. Creating a personal `/commit` skill won't override the project version—the developer must use a different skill name. Skills are the current approach for creating custom slash commands, having replaced the older `.claude/commands/` system (which still works for backwards compatibility). Skills add optional features: a directory for supporting files, frontmatter to control whether you or Claude invokes them, and the ability for Claude to load them automatically when relevant. Options C and D reference frontmatter features that don't exist; there is no `override` option and no username-based conditional logic in skill frontmatter.

Option-level rationale: - A) This is correct. Since project skills take precedence over personal skills with the same name, the developer must use a different skill name (like

/my-commit) in their personal ~/.claude/skills/ directory to ensure their custom version is accessible alongside the team's project skill. - B) This is incorrect. Project skills in .claude/skills/ take precedence over personal skills in ~/.claude/skills/ when they share the same name, so creating a personal /commit skill would be shadowed by the project's version and never invoked. - C) This is incorrect. There is no override: true frontmatter option in the skill system; this feature does not exist, so a personal skill cannot be configured to take precedence over a project skill with the same name. - D) This is incorrect. Skill frontmatter does not support username-based conditional logic, and modifying the shared project skill to handle individual developer preferences would unnecessarily complicate the team's shared configuration.

Question 39

Topic: Multi-Agent Orchestration | Difficulty: medium | Scenario: 3 | ID: 737

The document analysis agent discovers that two credible sources contain directly conflicting statistics on a key metric: one government report states 40% growth while an industry analysis states 12% growth. Both sources appear legitimate and the discrepancy could significantly affect the research conclusions. What's the most effective way for the document analysis agent to handle this?

- **A)** Halt analysis and escalate to the coordinator immediately, asking it to determine which source is authoritative before the agent continues processing remaining documents.
- **B)** Apply source credibility heuristics to select the most likely accurate figure, complete the analysis using that value, and include a footnote mentioning the discrepancy.
- **C)** Complete the document analysis with both figures included, explicitly annotate the conflict with source attribution, and let the coordinator decide how to reconcile before passing to synthesis.
- **D)** Include both figures in the analysis output without flagging them as conflicting, allowing the synthesis agent to determine which to use based on the broader research context.

Correct Answer: C - Complete the document analysis with both figures included, explicitly annotate the conflict with source attribution, and let the coordinator decide how to reconcile before passing to synthesis.

Explanation: Option C correctly applies separation of concerns: the document analysis agent completes its core task (analyzing documents) without blocking, preserves all information with explicit conflict annotation, and defers the reconciliation decision to the coordinator which has broader context. Option A creates unnecessary workflow blocking. Option B has the wrong agent making credibility judgments that affect which data gets used. Option D risks the conflict being missed due to lack of annotation and inappropriately delegates the decision to the synthesis agent.

Option-level rationale: - A) Halting analysis and escalating immediately creates unnecessary workflow blocking, preventing the agent from completing its core document analysis task. The agent should finish processing all documents and flag the conflict rather than stopping work to wait for a coordinator decision. - B) Having the document analysis agent apply credibility heuristics to select one figure oversteps its role, as it lacks the broader context needed to make authoritative credibility judgments. This approach also risks losing important information by relegating the discrepancy to a footnote rather than treating it as a significant finding requiring reconciliation. - C) This is the most effective approach because it respects separation of concerns: the document analysis agent completes its primary task without blocking, preserves both conflicting data points with explicit source attribution, and appropriately defers the reconciliation decision to the coordinator, which has the broader context needed to resolve the conflict. - D) Including both figures without flagging the conflict risks the discrepancy being overlooked by downstream agents, and it inappropriately delegates the reconciliation decision to the synthesis agent rather than the coordinator. Explicit conflict annotation is essential to ensure significant data discrepancies are properly addressed in the workflow.

Question 40

Topic: Tool Selection Reliability | Difficulty: medium | Scenario: 1 | ID: 739

Production logs show the agent sometimes selects `get_customer` when `lookup_order` would be more appropriate, particularly for ambiguous requests like “I

need help with my recent purchase." You decide to add few-shot examples to your system prompt to improve tool selection. Which approach will most effectively address this issue?

- **A)** Add 10-15 examples of clear, unambiguous requests that demonstrate correct tool selection for each tool's typical use cases.
- **B)** Add 4-6 examples targeting ambiguous scenarios, each showing reasoning for why one tool was chosen over plausible alternatives.
- **C)** Add examples grouped by tool—all `get_customer` scenarios together, then all `lookup_order` scenarios.
- **D)** Add explicit "use when" and "do not use when" guidelines in each tool's description covering the ambiguous cases.

Correct Answer: B - Add 4-6 examples targeting ambiguous scenarios, each showing reasoning for why one tool was chosen over plausible alternatives.

Explanation: Option B is most effective because few-shot examples that target the specific failure mode (ambiguous requests) and include explicit reasoning help the model understand the decision-making process for edge cases. Worked examples demonstrating comparative tool selection are more effective than declarative rules alone for nuanced decisions. Option A focuses on unambiguous cases where there's no problem. Option C organizes examples in a way that doesn't help with comparative tool selection decisions. Option D provides helpful guidelines, but static tool descriptions are less effective than worked examples for teaching nuanced edge-case reasoning—the model needs to see the actual decision process, not just rules about when to use each tool.

Option-level rationale: - A) Providing examples of clear, unambiguous requests addresses cases where the model already performs well, not the specific failure mode of ambiguous requests. This approach wastes few-shot budget on easy cases and does nothing to teach the model how to reason through the tricky scenarios causing errors. - B) Targeting few-shot examples at the specific ambiguous scenarios where errors occur, with explicit reasoning about why one tool is preferred over another, directly teaches the model the comparative decision-making process it needs for edge cases. This approach is the most effective because worked examples demonstrating reasoning are better than declarative rules for nuanced tool selection. - C) Grouping examples by tool encourages the model to learn each tool's use cases in isolation, which fails to teach comparative reasoning about when to choose one tool over another for ambiguous requests. The

problem specifically involves distinguishing between tools, so examples need to contrast tools rather than present them separately. - D) Adding explicit usage guidelines to tool descriptions can help, but static declarative rules are less effective than worked examples for teaching nuanced edge-case reasoning. The model benefits more from seeing the actual decision process in context than from reading abstract rules about when to use or avoid each tool.

Question 41

Topic: Self-Evaluation Patterns | Difficulty: medium | Scenario: 1 | ID: 744

Production metrics show that when your agent resolves complex cases involving billing disputes or multi-order returns, customer satisfaction scores are 15% lower than for simple cases—even when the resolution is technically correct. Root cause analysis reveals the agent provides accurate resolutions but inconsistently explains the reasoning: sometimes omitting relevant policy details, other times missing timeline information or next steps. The specific context gaps vary by case. You want to improve resolution quality without adding human review overhead. Which approach is most effective?

- **A)** Add a self-critique step where the agent evaluates its draft response for completeness—ensuring it addresses the customer’s concern, includes relevant context, and anticipates follow-up questions.
- **B)** Implement few-shot examples in the system prompt showing complete resolution explanations for five common complex case types, demonstrating how to include policy context, timelines, and next steps.
- **C)** Increase the model tier from Haiku to Sonnet for complex cases, routing based on detected case complexity.
- **D)** Add a confirmation step where the agent asks “Does this fully address your concern?” before closing, letting customers request additional information if needed.

Correct Answer: A - Add a self-critique step where the agent evaluates its draft response for completeness—ensuring it addresses the customer’s concern, includes relevant context, and anticipates follow-up questions.

Explanation: Self-critique (evaluator-optimizer pattern) directly addresses the root cause: inconsistent inclusion of explanation elements. By having the agent evaluate its own response against specific criteria before presenting it, you catch case-specific gaps that vary with each situation. Few-shot examples (B) help with consistent patterns but cannot cover highly variable gaps. Model tier upgrades (C) don't address the structural issue—the resolutions are already accurate, just incompletely explained. Confirmation steps (D) shift the burden to customers rather than improving the agent's output quality.

Option-level rationale: - A) A self-critique step (evaluator-optimizer pattern) directly addresses the root cause of inconsistent explanation completeness by having the agent evaluate its own draft against specific criteria—such as policy context, timelines, and next steps—before presenting it. This catches case-specific gaps that vary across different complex scenarios without requiring human review. - B) While few-shot examples can demonstrate ideal response structure for common case types, they cannot adequately cover the highly variable context gaps that differ from case to case. This approach works better for consistent, predictable patterns rather than the diverse omissions described in the scenario. - C) Upgrading the model tier does not address the structural issue, since the agent is already producing technically correct resolutions—the problem is inconsistent inclusion of explanatory context, not insufficient reasoning capability. A more capable model without a process change would likely exhibit the same inconsistency. - D) Adding a customer confirmation step shifts the burden of identifying missing information onto the customer rather than proactively improving the agent's output quality. This increases customer effort and friction, which is likely to further harm satisfaction scores rather than improve them.

Question 42

Topic: Conversation Context Management | Difficulty: hard | Scenario: 1 | ID: 749

Your support agent uses progressive summarization—when context reaches 70% capacity, older turns are summarized while recent ones remain verbatim. Production logs reveal a pattern: customers reference specific amounts (“the 15% discount I mentioned”), but the agent responds with incorrect values. Investigation shows these details were stated 20+ turns ago and got condensed into vague summaries like “discussed promotional pricing.” What’s the most effective fix?

- **A)** Extract transactional facts (amounts, dates, order numbers) into a persistent “case facts” block included in each prompt, outside the summarized history.
- **B)** Revise the summarization prompt to explicitly preserve all numerical values, percentages, dates, and customer-stated expectations verbatim.
- **C)** Increase the summarization threshold from 70% to 85% capacity so conversations have more room before summarization triggers.
- **D)** Store full conversation history externally and implement retrieval to search it when the agent detects reference phrases like “as I mentioned.”

Correct Answer: A - Extract transactional facts (amounts, dates, order numbers) into a persistent “case facts” block included in each prompt, outside the summarized history.

Explanation: Option A is correct because structured extraction addresses the root cause: summarization is inherently lossy for precise details. By extracting transactional facts (amounts, dates, order numbers) into a persistent block before summarization, critical information is preserved reliably and available in every prompt. Option B fails because LLMs don’t reliably follow preservation instructions under context pressure—you can’t “prompt your way out” of summarization being lossy. Option C only delays the problem without solving it; longer conversations will still lose early details. Option D is over-engineered and fragile—reference phrase detection is unreliable, and it doesn’t help when the agent proactively uses wrong values without the customer explicitly referencing them.

Option-level rationale: - A) Extracting transactional facts (amounts, dates, order numbers) into a persistent “case facts” block addresses the root cause: summarization is inherently lossy for precise details. By preserving critical information in a structured block outside the summarized history, these facts remain reliably available in every prompt regardless of how many turns are summarized. - B) Revising the summarization prompt to preserve numerical values sounds reasonable but is unreliable in practice—LLMs don’t consistently follow such preservation instructions under context pressure, meaning summarization will still be lossy for precise details. You cannot reliably “prompt your way out” of the fundamental information loss that summarization introduces. - C) Raising the summarization threshold from 70% to 85% only delays when summarization occurs rather than solving the underlying problem. Longer conversations will still eventually trigger summarization and lose the same critical details, making this a temporary workaround rather than a fix. - D) Storing full history externally with retrieval triggered by reference phrases is over-engineered and fragile, since detecting when a customer is referencing earlier details is unreliable. It also fails when the agent

proactively uses incorrect values without the customer explicitly referencing prior statements.

Question 43

Topic: Error Propagation | Difficulty: easy | Scenario: 3 | ID: 750

The document analysis subagent encounters a corrupted PDF file it cannot parse. When designing the system's error handling, what is the most effective way to handle this failure?

- **A)** Return the error with context to the coordinator agent, letting it decide how to proceed.
- **B)** Silently skip the corrupted document and continue processing other files to avoid interrupting the workflow.
- **C)** Throw an exception that terminates the entire research workflow.
- **D)** Automatically retry parsing the document three times with exponential backoff before reporting failure.

Correct Answer: A - Return the error with context to the coordinator agent, letting it decide how to proceed.

Explanation: Returning errors with context to the coordinator is the correct approach because it allows the coordinator agent to make informed decisions about how to proceed—whether to skip the document, try an alternative, or inform the user. Silent failures (B) hide critical information and undermine report completeness. Terminating the entire workflow (C) is disproportionate for a single file issue. Retrying (D) is pointless for corrupted files since corruption is a permanent failure, not a transient one that might resolve.

Option-level rationale: - A) Returning the error with context to the coordinator agent is the most effective approach because it enables the coordinator to make an informed decision—such as skipping the file, trying an alternative parsing method, or notifying the user—while maintaining visibility into the failure. - B) Silently skipping the corrupted document hides critical information from the coordinator and the user, potentially undermining the completeness and accuracy of the final research output without

anyone being aware of the gap. - C) Terminating the entire research workflow due to a single corrupted file is a disproportionate response that sacrifices all progress and prevents the system from processing the remaining valid documents. - D) Retrying with exponential backoff is a strategy suited for transient failures such as network timeouts, but file corruption is a permanent condition that will not resolve on its own, making repeated attempts wasteful and ineffective.

Question 44

Topic: Ambiguous Result Handling | Difficulty: medium | Scenario: 1 | ID: 751

Your `get_customer` tool returns all matches when searching by name. Claude currently picks the customer with the most recent order when multiple results are returned, but production data shows this causes 15% of multi-match cases to proceed with the wrong customer account. How should you address this?

- **A)** Instruct Claude to ask for an additional identifier (email, phone, or order number) when `get_customer` returns multiple matches, before taking any customer-specific action.
- **B)** Modify `get_customer` to return only the single most likely match based on a ranking algorithm, simplifying Claude's decision by eliminating ambiguous results.
- **C)** Add few-shot examples showing Claude how to use conversational context (products mentioned, dates referenced) to infer the correct customer without requiring clarification.
- **D)** Implement a confidence scoring system that proceeds automatically above 85% confidence and prompts for clarification below that threshold.

Correct Answer: A - Instruct Claude to ask for an additional identifier (email, phone, or order number) when `get_customer` returns multiple matches, before taking any customer-specific action.

Explanation: When tool results are ambiguous (multiple matches), asking the user for a disambiguating identifier is the most reliable approach. The user has definitive knowledge of their own identity, making one extra conversation turn worth avoiding the 15% error rate. Option B hides ambiguity from Claude, removing its ability to handle

edge cases. Option C relies on unreliable contextual inference. Option D uses arbitrary confidence thresholds that aren't well-calibrated for accuracy.

Option-level rationale: - A) Asking the user for an additional identifier (such as email, phone, or order number) is the most reliable way to disambiguate multiple matches, since the user has definitive knowledge of their own identity. One extra conversational turn is a small cost to eliminate the 15% error rate caused by incorrect customer selection. - B) Modifying the tool to return only a single best-guess match hides the ambiguity from Claude, removing its ability to recognize and handle cases where multiple customers share the same name. This approach may reduce errors in some cases but eliminates transparency and prevents proper disambiguation when the ranking algorithm is wrong. - C) Relying on conversational context clues like mentioned products or dates to infer the correct customer is unreliable, as these signals may be absent, ambiguous, or misleading. This approach trades one form of guessing for another without addressing the fundamental problem of needing definitive identification. - D) Using an arbitrary confidence threshold to decide when to auto-proceed versus ask for clarification is difficult to calibrate accurately and still allows errors whenever the confidence score is misleadingly high. This approach adds complexity without guaranteeing correct customer identification, unlike directly asking the user for a disambiguating identifier.

Question 45

Topic: Multi-Agent Orchestration | Difficulty: easy | Scenario: 3 | ID: 753

The web search and document analysis agents have both completed their tasks and returned findings to the coordinator. What is the appropriate next step for producing an integrated research output?

- **A)** The coordinator concatenates the raw outputs from both agents and returns them as the final result
- **B)** The coordinator passes both sets of findings to the synthesis agent for unified integration
- **C)** Each agent directly sends its findings to the report generation agent, bypassing the coordinator

- **D)** The document analysis agent requests the web search results and merges them internally

Correct Answer: B - The coordinator passes both sets of findings to the synthesis agent for unified integration

Explanation: The orchestrator-workers pattern requires the coordinator to maintain control of the workflow by collecting results from subagents and passing them to the appropriate specialized component. Option B correctly uses the synthesis agent for its intended purpose—integrating findings into a coherent output. Option A fails because concatenation is not integration and produces incoherent results. Options C and D violate the coordination pattern by having agents bypass the coordinator or communicate peer-to-peer, which breaks centralized control and creates architectural problems.

Option-level rationale: - A) Simply concatenating raw outputs from both agents does not constitute meaningful integration, as it would produce an incoherent result lacking synthesis or unified analysis. The coordinator should leverage a specialized synthesis agent to produce a coherent, integrated research output. - B) This is correct because the orchestrator-workers pattern requires the coordinator to maintain centralized control by collecting results from subagents and routing them to the appropriate next component—in this case, the synthesis agent, which is specifically designed to unify and integrate findings into a coherent output. - C) Having agents communicate directly with the report generation agent bypasses the coordinator, violating the centralized control principle of the orchestrator-workers pattern. This creates architectural problems by introducing unmanaged peer-to-peer communication and removing the coordinator’s ability to oversee the workflow. - D) Having one worker agent request results from another and merge them internally introduces peer-to-peer communication that breaks the centralized coordination pattern. The coordinator, not individual agents, should be responsible for collecting and routing all intermediate results through the workflow.

Question 46

The web search subagent returns results for only 3 of 5 requested source categories (competitor websites and industry reports succeeded, but news archives and social media feeds timed out). The document analysis subagent successfully processed all provided documents. The synthesis subagent must now produce a findings summary from this mixed-quality input. What's the most effective error propagation strategy?

- **A)** Structure the synthesis output with coverage annotations indicating which findings are well-supported versus which topic areas have gaps due to unavailable sources.
- **B)** Have the synthesis subagent request the coordinator retry the timed-out sources with extended timeouts before proceeding, ensuring complete data coverage before synthesis begins.
- **C)** Proceed with synthesis using only the successful sources, generating output without indicating which data was unavailable.
- **D)** Have the synthesis subagent return an error to the coordinator indicating incomplete upstream data, triggering a full retry or task failure.

Correct Answer: A - Structure the synthesis output with coverage annotations indicating which findings are well-supported versus which topic areas have gaps due to unavailable sources.

Explanation: Option A is correct because it embodies graceful degradation with transparency—continuing with available data while propagating uncertainty information downstream. This allows the report generator and end users to make informed decisions about confidence levels. Option B violates separation of concerns (subagents shouldn't direct coordinator retry logic) and assumes retries will succeed. Option C hides critical information, producing potentially misleading research reports. Option D treats partial success as total failure, wasting completed work and providing no value when partial results would be useful.

Option-level rationale: - A) Structuring the synthesis output with coverage annotations embodies graceful degradation with transparency, allowing downstream consumers and end users to understand which findings are well-supported and which topic areas have gaps. This approach preserves the value of completed work while propagating uncertainty information so informed decisions can be made about confidence levels. - B) Having the synthesis subagent direct the coordinator to retry timed-out sources violates separation of concerns, as subagents should not dictate coordinator retry logic. Additionally, this approach assumes retries will succeed and unnecessarily delays

producing useful output from the data already available. - C) Proceeding with synthesis without indicating which data sources were unavailable hides critical information from downstream consumers, potentially producing misleading research reports. Users would have no way to know that certain topic areas lack coverage, undermining the trustworthiness of the findings. - D) Treating partial success as total failure by returning an error wastes all the successfully completed work from both the web search and document analysis subagents. This approach provides no value when partial results with appropriate caveats would still be useful to stakeholders.

Question 47

Topic: Multi-Agent Orchestration | Difficulty: medium | Scenario: 3 | ID: 766

When researching a broad topic, you observe that the web search agent and document analysis agent are both investigating the same subtopics, resulting in significant overlap in their findings. Token usage has nearly doubled without proportionally increasing the breadth or depth of research coverage. What's the most effective way to address this?

- **A)** Have the coordinator explicitly partition the research space before delegation, assigning distinct subtopics or source types to each agent
- **B)** Implement a shared state mechanism where agents log their current focus area, allowing other agents to dynamically avoid duplicating work in progress
- **C)** Allow both agents to complete their parallel work, then have the coordinator deduplicate overlapping findings before passing to the synthesis agent
- **D)** Convert to sequential execution where document analysis runs only after web search completes, using the web search findings as context to avoid duplication

Correct Answer: A - Have the coordinator explicitly partition the research space before delegation, assigning distinct subtopics or source types to each agent

Explanation: Option A is correct because coordinator-level task partitioning addresses the root cause (unclear boundaries) before work begins, preserves parallelism, and prevents wasted tokens. Option B adds unnecessary complexity with shared state and race conditions. Option C fails to address the stated concern—tokens are still wasted even if results are deduplicated afterward. Option D unnecessarily sacrifices parallelism when simple partitioning achieves the same deduplication benefit.

Option-level rationale: - A) Having the coordinator explicitly partition the research space before delegation is the most effective approach because it addresses the root cause—unclear task boundaries—before any work begins. This preserves the benefits of parallel execution while preventing duplicated effort and wasted tokens. - B) While a shared state mechanism could theoretically reduce overlap, it introduces unnecessary complexity, potential race conditions, and still allows some duplicated work before agents detect each other's focus areas. Proactive partitioning at the coordinator level is simpler and more reliable than reactive dynamic coordination between agents. - C) Deduplicating findings after both agents complete their work fails to address the core problem of wasted tokens, since both agents still perform redundant research in parallel. This approach only cleans up the output without reducing the unnecessary resource consumption that was identified as the concern. - D) Converting to sequential execution unnecessarily sacrifices the performance benefits of parallelism when the same deduplication goal can be achieved through upfront task partitioning. This approach also increases total execution time without offering meaningful advantages over proactive partitioning of the research space.

Question 48

Topic: Error Propagation | Difficulty: medium | Scenario: 3 | ID: 770

During a materials research task, the web search subagent queries three source categories with different outcomes: academic databases returned 15 relevant papers, industry reports returned "0 results found," and patent databases returned "Connection timeout." When designing error propagation to the coordinator, what approach enables the best recovery decisions?

- **A)** Report both the timeout and "0 results" as failures requiring coordinator intervention.
- **B)** Have the subagent retry transient failures internally and only report persistent errors.
- **C)** Distinguish access failures (timeout) needing retry decisions from valid empty results ("0 results") representing successful queries.
- **D)** Aggregate outcomes into a single success rate metric (e.g., "67% source coverage") with detailed logs available on request.

Correct Answer: C – Distinguish access failures (timeout) needing retry decisions from valid empty results (“0 results”) representing successful queries.

Explanation: Option C correctly identifies that a timeout (query didn’t complete) is semantically different from “0 results” (query succeeded but found nothing). In research contexts, knowing a source has no relevant data is informative, while a timeout means the data is unknown. This distinction enables appropriate decisions: retry access failures but accept empty results as valid findings. Option A incorrectly treats valid empty results as failures, causing wasteful retries. Option B hides retry context from the coordinator, limiting resource budget management. Option D destroys actionable information through aggregation.

Option-level rationale: – A) Treating both the timeout and the ‘0 results’ outcome as failures requiring coordinator intervention is incorrect because ‘0 results’ represents a successful query that simply found no matching data, not a failure. This approach would cause unnecessary retries and waste resources on sources that have already been definitively queried. – B) Having the subagent retry transient failures internally without reporting them hides important context from the coordinator, which limits its ability to manage resource budgets and make informed decisions about retry strategies. The coordinator needs visibility into access failures to appropriately allocate time and resources across the workflow. – C) This is correct because a timeout (access failure) and ‘0 results’ (valid empty result) are semantically distinct outcomes requiring different responses. Distinguishing them enables the coordinator to retry the timed-out patent database while accepting the empty industry report results as a valid and informative finding. – D) Aggregating outcomes into a single success rate metric like ‘67% source coverage’ destroys the actionable detail needed for recovery decisions, since it obscures whether the missing 33% is due to transient access failures or legitimately empty results. The coordinator cannot make appropriate retry or escalation decisions without understanding the nature of each individual outcome.

Question 49

Topic: Tool Distribution | Difficulty: medium | Scenario: 3 | ID: 771

Production logs reveal a consistent pattern: requests to “analyze the quarterly report I uploaded” are routed to the web search agent 45% of the time instead of the document

analysis agent. Examining the tool definitions, you find the web search agent has an `analyze_content` tool described as “analyzes content and extracts key information,” while the document analysis agent has an `analyze_document` tool described as “analyzes documents and extracts key information.” How should you address this misrouting?

- **A)** Rename the web search tool to `extract_web_results` and update its description to “processes and returns information retrieved from web searches and URLs.”
- **B)** Add a pre-routing classifier that determines whether the user is referencing uploaded files or web content before the coordinator makes delegation decisions.
- **C)** Expand the document analysis tool’s description to include example use cases like “Use for uploaded PDFs, Word documents, and spreadsheets” while leaving the web search tool unchanged.
- **D)** Add few-shot examples to the coordinator’s prompt showing correct routing: “User uploads quarterly report → document analysis agent” and “User asks about a webpage → web search agent.”

Correct Answer: A - Rename the web search tool to `extract_web_results` and update its description to “processes and returns information retrieved from web searches and URLs.”

Explanation: Option A directly addresses the root cause: the semantic overlap between tool names and descriptions. Renaming `analyze_content` to `extract_web_results` and using “processes and returns information retrieved from web searches” completely eliminates the ambiguous “analyze” terminology that was confusing the coordinator. Option B over-engineers with an unnecessary classifier when the fix is simply clearer descriptions. Option C only fixes one side of the overlap, leaving the web search tool ambiguous. Option D treats the symptom through prompt workarounds rather than fixing the underlying tool definition problem.

Option-level rationale: - A) Renaming the web search tool to `extract_web_results` and updating its description to clearly reference web searches and URLs directly addresses the root cause by eliminating the semantic overlap between the two tools’ names and descriptions. This makes each tool’s purpose unambiguous, allowing the coordinator to correctly distinguish between document analysis and web search tasks. - B) Adding a pre-routing classifier over-engineers the solution by introducing an additional component when the core issue is simply ambiguous tool definitions. Fixing the tool names and descriptions is a simpler, more maintainable approach that

addresses the root cause rather than adding architectural complexity. - C) Expanding only the document analysis tool's description addresses just one side of the semantic overlap, leaving the web search tool's ambiguous `analyze_content` name and generic description intact. This partial fix may reduce misrouting but does not fully resolve the confusion since the web search tool still appears relevant for general content analysis tasks. - D) Adding few-shot examples to the coordinator's prompt is a workaround that treats the symptom rather than fixing the underlying problem of ambiguous tool definitions. This approach is fragile and may not generalize well to varied phrasings of similar requests, whereas clarifying the tool definitions eliminates the confusion at its source.

Question 50

Topic: Error Propagation | Difficulty: medium | Scenario: 3 | ID: 779

The document analysis subagent frequently encounters failures when processing PDF files—some have corrupted sections causing parsing exceptions, others are password-protected, and occasionally the parsing library times out on large files. Currently, any exception immediately terminates the subagent and returns an error to the coordinator, which must decide whether to retry, skip the document, or fail the entire research task. This is causing excessive coordinator involvement in routine error handling. What's the most effective architectural improvement?

- **A)** Have the subagent implement local recovery for transient failures and only propagate errors it cannot resolve to the coordinator, including what was attempted and any partial results obtained.
- **B)** Configure the subagent to always return partial results with success status, embedding error details in metadata. The coordinator treats all responses as successful and filters problematic items during synthesis.
- **C)** Have the coordinator validate all documents before dispatching to the subagent, rejecting documents likely to cause failures to ensure the subagent only receives processable files.
- **D)** Create a dedicated error-handling agent that monitors all subagent failures via a shared queue and makes recovery decisions independently, dispatching retry commands directly to subagents.

Correct Answer: A - Have the subagent implement local recovery for transient failures and only propagate errors it cannot resolve to the coordinator, including what was attempted and any partial results obtained.

Explanation: Option A is correct because it follows the principle of handling errors at the lowest level capable of resolving them. The subagent handles transient failures locally (reducing coordinator burden) while escalating only unresolvable issues with context for informed decisions. Option B masks errors as success, hiding problems rather than solving them. Option C increases coordinator burden by adding validation responsibilities. Option D adds unnecessary architectural complexity and coordination overhead.

Option-level rationale: - A) Implementing local recovery for transient failures within the subagent follows the principle of handling errors at the lowest level capable of resolving them. This reduces excessive coordinator involvement while still escalating truly unresolvable issues with full context, including what recovery was attempted and any partial results obtained. - B) Masking errors as success and embedding error details in metadata hides real problems from the coordinator rather than solving them. This approach risks silent data quality issues during synthesis and removes the coordinator's ability to make informed decisions about genuinely unresolvable failures. - C) Pre-validating all documents at the coordinator level increases the coordinator's burden rather than reducing it, and shifts routine error-handling responsibilities upward. Additionally, pre-validation cannot reliably predict all failure modes such as timeouts on large files or subtle corruption detected only during full parsing. - D) Introducing a dedicated error-handling agent adds unnecessary architectural complexity, an additional coordination layer, and a shared queue dependency. This approach distributes error-handling logic across multiple components rather than resolving failures at the most appropriate level—the subagent itself.

Question 51

Topic: Tool Distribution | Difficulty: medium | Scenario: 3 | ID: 781

When designing the system, you gave the document analysis agent access to a general-purpose `fetch_url` tool so it could load documents from URLs. Production logs reveal this agent now frequently fetches search engine result pages to conduct ad-hoc web

searches—behavior that should route through the web search agent. This causes inconsistent results. What's the most effective fix?

- **A)** Replace `fetch_url` with a `load_document` tool that validates URLs point to document formats.
- **B)** Remove `fetch_url` from the document analysis agent and route all URL loading through the coordinator to the web search agent.
- **C)** Add instructions to the document analysis agent's prompt clarifying it should only use `fetch_url` for loading document URLs, not searching.
- **D)** Implement filtering that blocks `fetch_url` calls to known search engine domains while allowing other URLs.

Correct Answer: A - Replace `fetch_url` with a `load_document` tool that validates URLs point to document formats.

Explanation: Option A is correct because it addresses the root cause by constraining the tool's capability at the interface level. A `load_document` tool with format validation makes the undesired search behavior impossible rather than merely discouraged—following the principle of least privilege. Option B removes useful capability and adds latency for legitimate document loading. Option C relies on prompt instructions, which provide unreliable enforcement since LLMs can ignore instructions when the tool technically permits the behavior. Option D uses a blocklist approach that requires ongoing maintenance and can never be complete, addressing symptoms rather than the underlying design flaw.

Option-level rationale: - A) Replacing the general-purpose tool with a document-specific tool that validates URLs point to document formats addresses the root cause by constraining capability at the interface level. This follows the principle of least privilege, making the undesired search behavior impossible rather than merely discouraged. - B) Removing URL fetching entirely from the document analysis agent and routing all requests through the coordinator eliminates useful capability and adds unnecessary latency for legitimate document loading. This over-corrects the problem when the agent legitimately needs to load documents from URLs. - C) Relying on prompt instructions to constrain tool usage provides unreliable enforcement, since LLMs can ignore or misinterpret instructions when the tool technically permits the undesired behavior. This approach addresses symptoms through soft guidance rather than enforcing constraints at the tool level. - D) Blocking known search engine domains uses a blocklist approach that requires ongoing maintenance and can never be comprehensive, as new search

engines or alternative URLs can bypass the filter. This addresses symptoms rather than fixing the underlying design flaw of an overly permissive tool.

Question 52

Topic: Long Context Position Effects | Difficulty: medium | Scenario: 3 | ID: 795

Production monitoring reveals inconsistent synthesis quality. When aggregated results total ~75K tokens, the synthesis agent reliably cites information from the first 15K tokens (web search headlines and snippets) and the final 10K tokens (document analysis conclusions), but frequently omits critical findings that appear in the middle 50K tokens—even when those findings directly address the research question. How should you restructure the aggregated input?

- **A)** Place a key findings summary at the beginning of the aggregated input and organize detailed results with explicit section headers for easier navigation.
- **B)** Implement rotation that alternates which subagent's results appear first across different research tasks, ensuring both sources receive primacy positioning equally over time.
- **C)** Summarize all subagent outputs to under 20K tokens total before aggregation, ensuring content stays within the model's reliable processing range.
- **D)** Stream subagent results to the synthesis agent incrementally, processing web search results first to completion before introducing document analysis findings.

Correct Answer: A - Place a key findings summary at the beginning of the aggregated input and organize detailed results with explicit section headers for easier navigation.

Explanation: LLMs exhibit the "lost in the middle" phenomenon where information at the beginning and end of long contexts is better recalled than middle content. Option A directly addresses this by placing key findings summaries in the primacy position and using explicit section headers for navigation. Option B only addresses fairness across tasks, not quality for any individual task. Option C risks losing critical findings during summarization. Option D prevents holistic synthesis by processing sources sequentially rather than allowing the model to see all inputs together.

Option-level rationale: - A) Placing a key findings summary at the beginning leverages the primacy effect, ensuring critical information occupies the most reliably attended position. Adding explicit section headers throughout the aggregated input helps the model navigate and attend to middle-section content, directly mitigating the 'lost in the middle' phenomenon. - B) Rotating which subagent's results appear first only redistributes which findings get lost across different tasks rather than solving the core problem. This approach treats fairness between sources rather than ensuring reliable quality for any individual synthesis task. - C) Aggressively summarizing all outputs to under 20K tokens risks discarding critical detailed findings during the compression process. While shorter context would avoid the lost-in-the-middle effect, the information loss from summarization could be worse than the original problem. - D) Streaming results sequentially and processing them one at a time prevents the synthesis agent from performing holistic cross-source analysis. This approach fragments the synthesis task rather than addressing the attention distribution issue within a single comprehensive context.

Question 53

Topic: Multi-Instance Verification | Difficulty: medium | Scenario: 16 | ID: 830

Your team uses Claude Code to generate code suggestions, but you notice a pattern: subtle issues—performance optimizations that break edge cases, cleanups that change behavior unexpectedly—only surface when a different team member reviews the PR. Claude's reasoning during generation shows it considered these cases but concluded its approach was correct. Which approach directly addresses the root cause of this self-review limitation?

- **A)** Have a second, independent Claude Code instance review the changes without seeing the generator's reasoning.
- **B)** Add explicit self-review instructions to the generation prompt, asking Claude to critique its own suggestions before finalizing output.
- **C)** Enable extended thinking mode for the generation pass, allowing more thorough deliberation before producing suggestions.
- **D)** Include comprehensive test files and documentation in the prompt context so Claude better understands expected behavior during generation.

Correct Answer: A - Have a second, independent Claude Code instance review the changes without seeing the generator's reasoning.

Explanation: Running a second, independent Claude instance without the generator's reasoning addresses the root cause of self-review blind spots. LLMs exhibit confirmation bias when reviewing their own work within the same context. By giving the reviewer only the code and proposed changes (not the reasoning behind them), it evaluates changes from a fresh perspective. Option B fails because self-review within the same context carries the same biases. Options C and D improve generation quality but don't address the fundamental self-review limitation—the question explicitly states Claude considered these cases during generation and concluded its approach was correct, so more thinking time (C) or more context (D) won't help catch issues Claude has already rationalized.

Option-level rationale: - A) Using a second, independent Claude Code instance without access to the generator's reasoning directly addresses the root cause by eliminating confirmation bias. This fresh perspective mirrors the benefit of human peer review, where a different team member catches issues the original author rationalized away. - B) Asking Claude to critique its own suggestions within the same context does not address the root cause, because the same confirmation bias that led it to conclude its approach was correct will persist during self-review. The question explicitly states Claude already considered these cases and rationalized its decisions, so additional self-critique in the same context will likely reach the same conclusions. - C) Enabling extended thinking mode gives Claude more deliberation time during generation, but the question states Claude already considered the edge cases and concluded its approach was correct. More thinking time does not resolve the fundamental self-review limitation where the model has already rationalized its decisions. - D) Providing more context such as test files and documentation improves the quality of generation input, but the question specifies Claude's reasoning already considered these cases and still concluded its approach was correct. This means the issue is not a lack of information but rather a self-review blind spot that additional context cannot fix.

Question 54

Topic: Classification Consistency | Difficulty: medium | Scenario: 16 | ID: 839

Your automated code review system shows inconsistent severity ratings—similar issues like null pointer risks receive “critical” severity in some PRs but only “medium” in others. Developer trust is declining because teams can’t predict which findings require immediate attention. What’s the most effective way to improve severity consistency?

- **A)** Include explicit severity criteria in your prompt with concrete code examples for each severity level
- **B)** Add a CLAUDE.md file that lists issue types and their default severities, instructing Claude to reference this mapping when assigning ratings
- **C)** Request that Claude include its reasoning for each severity assignment, then use that reasoning to manually calibrate and adjust ratings during review
- **D)** Modify the prompt to ask Claude to rate severity relative to other issues in the same PR, so the most severe issue is always marked critical and others rated proportionally

Correct Answer: A - Include explicit severity criteria in your prompt with concrete code examples for each severity level

Explanation: Explicit severity criteria with concrete code examples directly address the root cause of inconsistency: ambiguous interpretation of what each severity level means. This is a proven prompt engineering technique for improving classification reliability. Option B’s issue-type-to-severity mapping loses important context (the same issue type may warrant different severities based on code path). Option C shifts the burden to manual review, defeating automation. Option D’s relative ratings would make a single minor issue “critical” if it’s the only finding, introducing more inconsistency rather than less.

Option-level rationale: - A) Including explicit severity criteria with concrete code examples directly addresses the root cause of inconsistency by removing ambiguity about what each severity level means. This is a proven prompt engineering technique that gives the model clear reference points for classification, leading to more reliable and predictable severity assignments. - B) A static issue-type-to-severity mapping loses important context, since the same issue type (e.g., a null pointer risk) may warrant different severities depending on factors like code path, exposure, or criticality of the affected component. This rigid approach oversimplifies severity assignment and can lead to inaccurate ratings. - C) Requesting reasoning and then manually calibrating ratings shifts the burden to human reviewers, which defeats the purpose of automated code review and does not scale. While transparency in reasoning is valuable, it does not

directly solve the consistency problem in the automated system itself. - D) Rating severity relative to other issues within the same PR means that a single minor issue would be marked "critical" if it's the only finding, and truly critical issues could be downgraded if multiple severe issues exist. This relative approach introduces even more inconsistency rather than establishing a stable, absolute severity standard.

Question 55

Topic: Structured Output | Difficulty: medium | Scenario: 16 | ID: 841

Your CI pipeline runs the Claude Code CLI (with `--print` mode) using CLAUDE.md to provide project context for code reviews, and developers generally find the reviews insightful. However, they report that integrating findings into your workflow is difficult—Claude produces narrative paragraphs that must be manually copied into PR comments. Your team wants to automatically post each finding as a separate inline PR comment at the relevant code location, which requires structured data with file path, line number, severity, and suggested fix. What's the most effective approach?

- **A)** Add a "Review Output Format" section to CLAUDE.md with examples showing structured findings, so Claude learns the expected format from project context.
- **B)** Use CLI flags `--output-format json` and `--json-schema` to enforce structured findings, then parse output to post inline comments via the GitHub API.
- **C)** Include explicit formatting instructions in your review prompt requiring each finding to follow a parseable template like `[FILE:path] [LINE:n]`
`[SEVERITY:level] ...`.
- **D)** Keep the narrative review format but add a summarization step that uses Claude to generate a structured JSON summary of the findings.

Correct Answer: B - Use CLI flags `--output-format json` and `--json-schema` to enforce structured findings, then parse output to post inline comments via the GitHub API.

Explanation: The Claude Code CLI supports `--output-format json` with `--json-schema` to guarantee well-formed structured output that can be reliably parsed for automation. Options A and C rely on prompt-based formatting which LLMs follow

inconsistently—fine for human consumption but unreliable for automated pipelines requiring specific fields. Option D adds unnecessary complexity and cost with a second LLM call when the problem is solved natively with structured output enforcement via CLI flags.

Option-level rationale: - A) Adding format examples to CLAUDE.md relies on prompt-based formatting guidance, which LLMs follow inconsistently. This approach is unreliable for automated pipelines that require specific structured fields like file path, line number, and severity to be present and parseable every time. - B) Using `--output-format json` with `--json-schema` enforces structured output at the CLI level, guaranteeing well-formed JSON with the required fields (file path, line number, severity, suggested fix) that can be reliably parsed and posted as inline PR comments via the GitHub API. This is the most effective approach because it leverages native CLI capabilities designed specifically for structured output enforcement. - C) Including explicit formatting instructions in the prompt to produce a parseable template is a prompt-engineering approach that LLMs may follow inconsistently, sometimes deviating from the expected format. For automated pipelines requiring reliable parsing to post inline comments, this approach is fragile compared to using built-in CLI flags that enforce structured output. - D) Adding a second LLM call to summarize narrative output into structured JSON introduces unnecessary complexity, additional cost, and another potential point of failure. The problem can be solved more directly and reliably by enforcing structured output in the original CLI invocation using built-in flags.

Question 56

Topic: Prompt Specificity | Difficulty: medium | Scenario: 16 | ID: 844

Your automated review analyzes comments and docstrings. The current prompt instructs Claude to “check that comments are accurate and up-to-date.” Findings frequently flag acceptable patterns (TODO markers, straightforward descriptions) while missing comments that describe behavior the code no longer implements. What change addresses the root cause of this inconsistent analysis?

- **A)** Specify explicit criteria: flag comments only when their claimed behavior contradicts actual code behavior

- **B)** Add few-shot examples of misleading comments to help the model recognize similar patterns in the codebase
- **C)** Filter out TODO, FIXME, and descriptive comment patterns before analysis to reduce noise
- **D)** Include git blame data so Claude can identify comments that predate recent code modifications

Correct Answer: A – Specify explicit criteria: flag comments only when their claimed behavior contradicts actual code behavior

Explanation: The root cause is that “accurate and up-to-date” lacks a definition of what constitutes a problem. Explicit criteria (A) directly address this by defining the target: semantic contradiction between a comment’s claim and actual code behavior. Few-shot examples (B) help the model recognize similar patterns to those shown, but pattern-matching won’t generalize to novel misleading comments—the model needs the underlying criterion to identify any contradiction. Filtering (C) addresses one symptom (false positives on TODOs) but ignores the false negative problem entirely. Git blame data (D) provides temporal context but doesn’t define what should actually be flagged—old comments can be accurate, and new comments can be wrong.

Option-level rationale: – A) Specifying explicit criteria—flag comments only when their claimed behavior contradicts actual code behavior—directly addresses the root cause by replacing the vague instruction with a precise definition of what constitutes a problem. This eliminates both false positives on acceptable patterns and false negatives on genuinely misleading comments. – B) While few-shot examples of misleading comments can help the model recognize similar patterns, this approach won’t generalize well to novel types of contradictions because it relies on pattern-matching rather than defining the underlying criterion the model should apply. – C) Filtering out TODO, FIXME, and descriptive comments before analysis only addresses one symptom (false positives) while completely ignoring the false negative problem of missing comments that describe behavior the code no longer implements. – D) Including git blame data provides temporal context about when comments were written, but comment age alone doesn’t determine accuracy—old comments can still be correct and new comments can be wrong—so this doesn’t define what should actually be flagged.

Question 57

Topic: Batch Processing | Difficulty: easy | Scenario: 16 | ID: 847

Your CI pipeline includes two Claude-powered code review modes: a `pre-merge-commit` hook that blocks PR merging until complete, and "deep analysis" that runs overnight, polls for batch completion, then posts detailed suggestions to the PR. You want to reduce API costs using the Message Batches API, which offers 50% cost savings but requires polling and may take up to 24 hours to complete. Which mode should use batch processing?

- **A)** Pre-merge-commit hook only
- **B)** Deep analysis only
- **C)** Both modes
- **D)** Neither mode

Correct Answer: B - Deep analysis only

Explanation: Deep analysis is the correct choice because it runs overnight and doesn't require immediate results, making it ideal for batch processing's up-to-24-hour latency in exchange for 50% cost savings. The Message Batches API works asynchronously: you submit requests, poll for completion status, then retrieve results from a JSONL file—there's no callback mechanism. This polling model fits perfectly with overnight jobs that can check periodically and post PR comments once complete. Pre-merge-commit hooks block PR merging and require timely completion, making them incompatible with batch processing's latency—even though most batches complete in under an hour, the system makes no timing guarantees. Using batch for both modes would break the pre-merge-commit workflow, and avoiding batch entirely wastes cost savings on the latency-tolerant deep analysis workload.

Option-level rationale: - A) Using batch processing for the pre-merge-commit hook is inappropriate because it blocks PR merging and requires timely completion, which is incompatible with the Message Batches API's potential latency of up to 24 hours and lack of timing guarantees. This would cause unacceptable delays in the development workflow. - B) Deep analysis is the ideal candidate for batch processing because it already runs overnight, tolerates latency, and uses a polling model to check for completion before posting results—perfectly matching the Message Batches API's asynchronous, poll-based design while capturing the 50% cost savings. - C) Applying

batch processing to both modes would break the pre-merge-commit hook workflow, since it requires near-immediate results to unblock PR merging, and the Message Batches API cannot guarantee timely completion. While deep analysis benefits from batching, the blocking nature of pre-merge hooks makes them incompatible. - D) Avoiding batch processing entirely wastes the opportunity to save 50% on API costs for the deep analysis workload, which is inherently latency-tolerant and already designed around an asynchronous polling pattern that aligns perfectly with the Message Batches API.

Question 58

Topic: False Positive Reduction | Difficulty: medium | Scenario: 16 | ID: 850

Analysis of your automated code review shows significant variation in false positive rates across finding categories. Security and correctness findings have an 8% false positive rate, performance findings have 18%, style and naming findings have 52%, and documentation findings have 48%.

Developer surveys indicate growing distrust—many have started dismissing findings without review because “half are wrong.” The high false positive categories are undermining confidence in the accurate categories. What approach best restores developer trust while improving the system?

- **A)** Temporarily disable high false positive categories (style, naming, documentation) and run only high-precision categories while improving prompts.
- **B)** Apply a uniform strictness reduction across all categories to bring the overall false positive rate to an acceptable level.
- **C)** Keep all categories but display a confidence score with each finding, letting developers decide which to investigate.
- **D)** Keep all categories enabled while adding few-shot examples to improve each category's accuracy over the coming weeks.

Correct Answer: A - Temporarily disable high false positive categories (style, naming, documentation) and run only high-precision categories while improving prompts.

Explanation: Option A is correct because it immediately stops trust erosion by removing high-FP categories while preserving the value of accurate ones (security, correctness, performance). A tool giving 50% wrong advice is worse than no tool—developers who've lost trust won't regain it while noisy findings continue. Option B damages working categories to compensate for broken ones. Option C shifts burden to developers without reducing noise, and those who already distrust the system won't trust confidence scores. Option D addresses the right problem but ignores urgency—trust will continue eroding during iteration.

Option-level rationale: - A) Temporarily disabling the high false positive categories (style, naming, documentation) immediately stops trust erosion by removing the noise that causes developers to dismiss all findings, while preserving the value of high-precision categories like security and correctness. This approach allows time to improve prompts for the problematic categories before re-enabling them, rebuilding trust through demonstrated accuracy. - B) Applying uniform strictness reduction across all categories would degrade the performance of already-accurate categories (like security and correctness at 8% false positive rate) to compensate for broken ones, sacrificing real value without effectively solving the trust problem in any category. - C) Displaying confidence scores shifts the evaluation burden onto developers without actually reducing noise, and developers who have already lost trust in the system are unlikely to trust its self-reported confidence scores either. - D) While improving prompts with few-shot examples addresses the root cause, keeping all categories enabled during the weeks-long improvement process allows trust to continue eroding as developers keep encountering the ~50% false positive rates that have already caused them to dismiss findings without review.

Question 59

Topic: Few-Shot Prompting | Difficulty: medium | Scenario: 16 | ID: 856

Your automated reviews identify valid issues but developers report the feedback isn't actionable. Findings say things like "complex ticket allocation logic" or "potential null pointer" without specifying what to change. When you add detailed instructions like "always include specific fix suggestions," the model still produces inconsistent output—

sometimes detailed, sometimes vague. What prompting technique would most reliably produce consistently actionable feedback?

- **A)** Further refine the instructions with more explicit requirements for each part of the feedback format (location, issue, severity, suggested fix)
- **B)** Add 3-4 few-shot examples showing the exact format you want: issue identified, code location, specific fix suggestion
- **C)** Implement a two-pass approach where one prompt identifies issues and a second prompt generates fixes, allowing specialization
- **D)** Expand the context window to include more of the surrounding codebase so the model has sufficient information to suggest specific fixes

Correct Answer: B - Add 3-4 few-shot examples showing the exact format you want: issue identified, code location, specific fix suggestion

Explanation: Few-shot examples are the most effective technique for achieving consistent output format when instructions alone produce variable results. The scenario explicitly shows that instruction refinement failed, making Option A a repetition of a failing approach. Option B works because examples demonstrate the exact pattern expected, which models follow more reliably than abstract instructions. Option C overcomplicates the system without addressing format consistency, and Option D misdiagnoses the problem—the model sometimes succeeds with current context, so more context won't fix format variability.

Option-level rationale: - A) The scenario already describes adding detailed instructions that failed to produce consistent output, so further refining instructions repeats the same failing approach. More explicit requirements are still abstract instructions, which models follow less reliably than concrete examples. - B) Few-shot examples are the most effective technique for achieving consistent output format when instructions alone produce variable results. Providing 3-4 examples showing the exact desired format (issue, location, specific fix) gives the model a concrete pattern to follow, which is more reliable than abstract instructions. - C) A two-pass approach adds architectural complexity without directly addressing the core problem of inconsistent output formatting. While specialization can help in some cases, it doesn't solve why the model sometimes produces vague feedback instead of actionable suggestions. - D) Expanding the context window misdiagnoses the problem—the model already sometimes produces detailed, actionable feedback with the current context, indicating

that insufficient information is not the root cause. The real issue is format inconsistency, which more context does not resolve.

Question 60

Topic: False Positive Reduction | Difficulty: hard | Scenario: 16 | ID: 864

Your automated code review averages 15 findings per pull request, with developers reporting a 40% false positive rate. The bottleneck is investigation time: developers must click into each finding to read Claude's reasoning before deciding whether to address or dismiss it. Your CLAUDE.md already contains comprehensive rules for acceptable patterns, and stakeholders have rejected any approach that filters findings before developer review. What change would best address the investigation time bottleneck?

- **A)** Configure Claude to only surface findings it assesses as high confidence, filtering out uncertain flags before developers see them
- **B)** Require Claude to include its reasoning and confidence assessment inline with each finding
- **C)** Add a post-processor that analyzes finding patterns and automatically suppresses those matching historical false positive signatures
- **D)** Categorize findings as "blocking issues" versus "suggestions" with tiered review requirements

Correct Answer: B - Require Claude to include its reasoning and confidence assessment inline with each finding

Explanation: Including reasoning and confidence inline directly addresses the stated bottleneck (investigation time from clicking into findings) while respecting the constraint that findings cannot be filtered before developer review. Option A violates the explicit constraint against filtering. Option C also filters/suppresses findings before review. Option D reorganizes workflow but doesn't reduce investigation time—developers still need to click into findings in whichever category they review to understand why Claude flagged them.

Option-level rationale: - A) Filtering out low-confidence findings before developers see them directly violates the stakeholder constraint that no findings should be filtered before developer review. This approach would reduce the number of findings but is explicitly disallowed by the stated requirements. - B) Including reasoning and confidence assessments inline with each finding directly addresses the investigation time bottleneck by allowing developers to quickly evaluate findings without clicking into each one separately. This approach respects the constraint against filtering, since all findings remain visible while making triage significantly faster. - C) Automatically suppressing findings that match historical false positive signatures is another form of filtering before developer review, which stakeholders have explicitly rejected. Even though it uses data-driven pattern matching, it still removes findings from the developer's view. - D) Categorizing findings into tiers reorganizes the review workflow but does not reduce the core investigation time problem, as developers still need to click into each finding to understand Claude's reasoning before deciding to address or dismiss it. This approach adds structure without addressing the root cause of the bottleneck.
